POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Casco sensorizzato

Progetto per il corso di Laboratorio di Tecnologie Elettroniche e Biosensori
Master in Biomedical Engineering - BTE

Gotti Michela	**10710943**	**Moiana Laura**	**10733956**
Luppi Eleonora	**10656120**	**Morotti Gianluca**	**10728379**
Manini Filippo	**10743546**	**Rosato Elena**	**10620037**

Relatrice:
Prof. Alessandra Angelucci

Anno accademico:
2024-2025

Abstract: Cyclist safety remains a critical issue, as shown by recent studies that highlight a growing number of accidents involving cyclists, especially in urban areas. To address this, a smart helmet equipped with multiple sensors has been developed, to prevent or mitigate the risk of accidents. The system is designed to detect distractions, both from the cyclist and surrounding vehicles, and send an alarm in case of an accident. A SparkFun Qwiic ToF Imager is used to monitor the distance between the cyclist and nearby vehicles, triggering an alarm if the distance becomes too short, indicating a potential collision. Additionally, to detect the cyclist's own distractions, an Inertial Measurement Unit (IMU) tracks head pitch and reminds the cyclist to pay attention. The IMU also plays a key role in fall detection by integrating data from the accelerometer, gyroscope, and magnetometer. This data is processed using a Kalman filter, which extracts relevant angles to detect falls. When a fall is detected, the system sends an alert via Bluetooth Low Energy (BLE) to the user's phone, that allows to make an emergency call to a pre-selected contacts and share the cyclist's precise GPS location. The hardware implementation includes a microcontroller, IMU, ToF sensors, battery, and wiring. The firmware manages the sensor data processing, including kalman filter, fall detection, cyclist attention and vehicle detection, while BLE ensures communication with the cyclist's smartphone. Moreover, the mobile application, developed using Android Studio, allows users to log in, make emergency calls, view GPS position, and view cycling statistics.

Key-words: smart helmet, cyclist safety, fall detection, accident prevention, emergency alert, IMU, TOF, BLE, wearable technology

1. Introduzione

La sicurezza stradale rappresenta da sempre una delle principali preoccupazioni della cittadinanza.
Nel corso degli anni, sono state prese diverse misure per contrastare questo problema, arrivando ad una significativa riduzione del numero di incidenti in strada. Tuttavia, lo studio condotto nel 2020 [1] dall' European Road Safety Observatory, relativo al periodo 2010-2018, evidenzia che, sebbene il numero di incidenti mortali sia in netto calo (21%), si registra un'eccezione per quelli che coinvolgono i ciclisti.
Questo fenomeno sottolinea la necessità di sviluppare soluzioni mirate a prevenire gli incidenti che li riguardano, puntando su una maggiore educazione alla sicurezza stradale. È inoltre indispensabile incentivare lo sviluppo di dispositivi capaci di allertare preventivamente il ciclista in caso di situazioni pericolose o imminenti rischi di incidenti.
Esaminando la natura di tali sinistri, si può innanzitutto identificare il luogo di accadimento: secondo lo stesso rapporto dell'UE, il 55% degli incidenti avviene in ambito urbano, mentre il restante 45% si verifica al di fuori delle città. Questo dato è ancora più rilevante nella regione Lombardia, dove, secondo un report della Regione Lombardia del 2018, : "Nel triennio 2014-2016, su un totale di 13.156 incidenti con biciclette, ben 12.078 si sono verificati in ambito urbano (quasi il 92%)" [2].
Come riportato da Istat[3], la principale causa di incidenti è la distrazione del conducente, confermata anche dal fatto che circa il 60% degli incidenti avviene in assenza di incroci, una statistica che potrebbe sembrare controintuitiva. Di conseguenza, i sorpassi rappresentano i momenti più critici per l'incolumità del ciclista.
Oltre alla prevenzione degli incidenti, un altro fattore determinante nella gravità dei sinistri ciclistici è il ritardo nell'intervento di soccorso. Studi recenti [4] hanno dimostrato che ridurre il tempo di intervento di soli sessanta secondi può aumentare del 6% le probabilità di sopravvivenza della persona coinvolta. Pertanto, diventa essenziale implementare un sistema efficace di rilevamento delle cadute, capace di inviare un allarme tempestivo in situazioni di emergenza.

2. Obiettivi

L'obiettivo di questo progetto è lo sviluppo di un casco sensorizzato capace di agire su più livelli per prevenire o ridurre il rischio di incidenti in ambito prettamente urbano (estendibile anche ad un contesto extraurbano con specifiche di progetto più stringenti). Considerando che la principale fonte di incidenti è la distrazione, lo sviluppo della sensoristica si è orientato verso la gestione di due componenti principali:

1. **Distrazione del conducente di altri veicoli** (o mancato rispetto delle norme stradali) monitorata attraverso l'utilizzo di un sensore di distanza SparkFun Qwiic ToF Imager in grado di rilevare la distanza tra i due mezzi e generare un allarme in caso di possibile collisione;
2. **Distrazione del ciclista stesso** monitorata attraverso l'osservazione dell' inclinazione della testa rispetto alla direzione di movimento;

Il dispositivo proposto include anche un sistema avanzato di **fall detection**, basato sull'uso di un'unità di misura inerziale (IMU), comprendente accelerometro, giroscopio e magnetometro. Questi sensori raccolgono dati sul movimento e l'orientamento del ciclista, i quali vengono successivamente elaborati mediante un algoritmo di rilevazione delle cadute. Il sistema è progettato per identificare un possibile incidente e inviare automaticamente un segnale di emergenza, che include la posizione geografica precisa del ciclista fornita dal GPS integrato nello smartphone dell'utente. L'allarme viene inviato a contatti predefiniti per richiedere assistenza immediata.

3. Hardware

L'hardware del dispositivo proposto si articola nelle seguenti unità:

1. **Microcontrollore:** è stato utilizzato lo SparkFun Pro nRF52840 Mini[5], una scheda di sviluppo compatta basata sul chip nRF52840 di Nordic Semiconductor. Questo chip integra un processore ARM Cortex-M4, noto per la sua potenza di calcolo combinata con un consumo energetico contenuto, che lo rende ideale per applicazioni che richiedono elevate performance e bassa dissipazione di energia. Inoltre, essendo dotato di un modulo Bluetooth 2.4GHz, è perfetto per progetti wireless come dispositivi wearable e sistemi di comunicazione a corto raggio, anche grazie al circuito di ricarica integrato per batterie LiPo, che la svincola dalla necessità di una fonte di alimentazione fissa. Uno degli aspetti distintivi di questo microcontrollore è la sua dimensione compatta, simile a quella dell'Arduino Pro Mini, ma con un numero maggiore di funzionalità. E' dotato infatti di una varietà di pin di input/output che garantiscono grande flessibilità nel progettare il sistema. Tra i principali pin disponibili vi sono quelli per alimentazione, UART, I2C, SPI, PWM, ADC e Real-time clock (RTC). Per una comprensione più dettagliata delle connessioni fisiche, è possibile fare riferimento allo schematico del dispositivo in Figura 1, che fornisce

un'illustrazione chiara dei pin e delle loro funzionalità.
Il microcontrollore nRF52840 è dotato di cinque timer a 32 bit che operano in modalità contatore cioè contano gli impulsi di clock fino a un valore massimo definito, dopodiché generano un evento (come un interrupt) o si azzerano. Questi timer sono utili per una vasta gamma di applicazioni, come la gestione di eventi in tempo reale, la generazione di segnali PWM (Pulse Width Modulation), o il controllo preciso di intervalli di tempo. Infatti, ogni timer ha la possibilità di generare interrupt che consentono di rispondere in modo rapido a eventi esterni senza la necessità di una gestione continua da parte della CPU, ottimizzando così l'efficienza del sistema. Questa funzionalità è particolarmente utile in applicazioni che richiedono una gestione in tempo reale di più eventi, come la gestione della comunicazione BLE o la raccolta di dati da più sensori ad intervalli specifici. La memoria del microcontrollore è composta da 1 MB di flash e 256 KB di RAM, offrendo ampi spazi per archiviare e gestione dati, perfetto per applicazioni che richiedono prestazioni elevate e capacità di elaborazione in tempo reale. La programmazione del microcontrollore avviene tramite l'interfaccia USB nativa del nRF52840, che consente non solo di alimentare la scheda e comunicarvi, ma anche di caricare il codice sviluppato. Questo può avvenire utilizzando ambienti di sviluppo come Arduino IDE, CircuitPython o C con l'SDK di Nordic, offrendo una notevole flessibilità a seconda delle preferenze del programmatore. La scheda supporta diversi protocolli di comunicazione, tra cui UART, I2C e SPI, rendendola estremamente versatile. Un aspetto che semplifica ulteriormente lo sviluppo è il connettore Qwiic, che permette di aggiungere facilmente sensori e moduli aggiuntivi senza necessità di saldature. Questa funzionalità è ideale per espandere il progetto in modo rapido e sicuro, riducendo il rischio di errori nelle connessioni hardware. Inoltre, la scheda è dotata di un LED utente, un pulsante di reset e un interruttore di accensione, che semplificano ulteriormente il processo di sviluppo e testing.
Nel seguente progetto il microcontrollore è stato programmato tramite **Arduino IDE**, che ha facilitato lo sviluppo grazie ad un'interfaccia semplificata e alla presenza di librerie già disponibili per il microcontrollore e per i sensori. Per la connessione di questi ultimi, è stato sfruttato il **connettore Qwiic**, che consente di collegare i diversi moduli in configurazione daisy chain (a cascata). La comunicazione tra il microcontrollore e i sensori avviene tramite il protocollo I2C, supportato nativamente dal nRF52840, che offre un'efficiente soluzione per la gestione a bassa potenza di più dispositivi contemporaneamente, evitando conflitti tra di essi, grazie al controllo del microcontrollore che agisce da master. Per la comunicazione con il telefono cellulare, è stato utilizzato il **Bluetooth Low Energy (BLE)**, permettendo di estrarre la posizione GPS, avvisare dell'avvenuta caduta e inviare l'allarme. L'utilizzo del BLE è stato scelto per la sua efficienza energetica, che si adatta perfettamente alle esigenze di un dispositivo wearable alimentato a batteria. Per l'alimentazione, infatti, viene utilizzata una **batteria LiPo**[4], che consente di utilizzare il dispositivo una volta scollegato dalla fonte di alimentazione USB, offrendo così autonomia e portabilità al sistema.

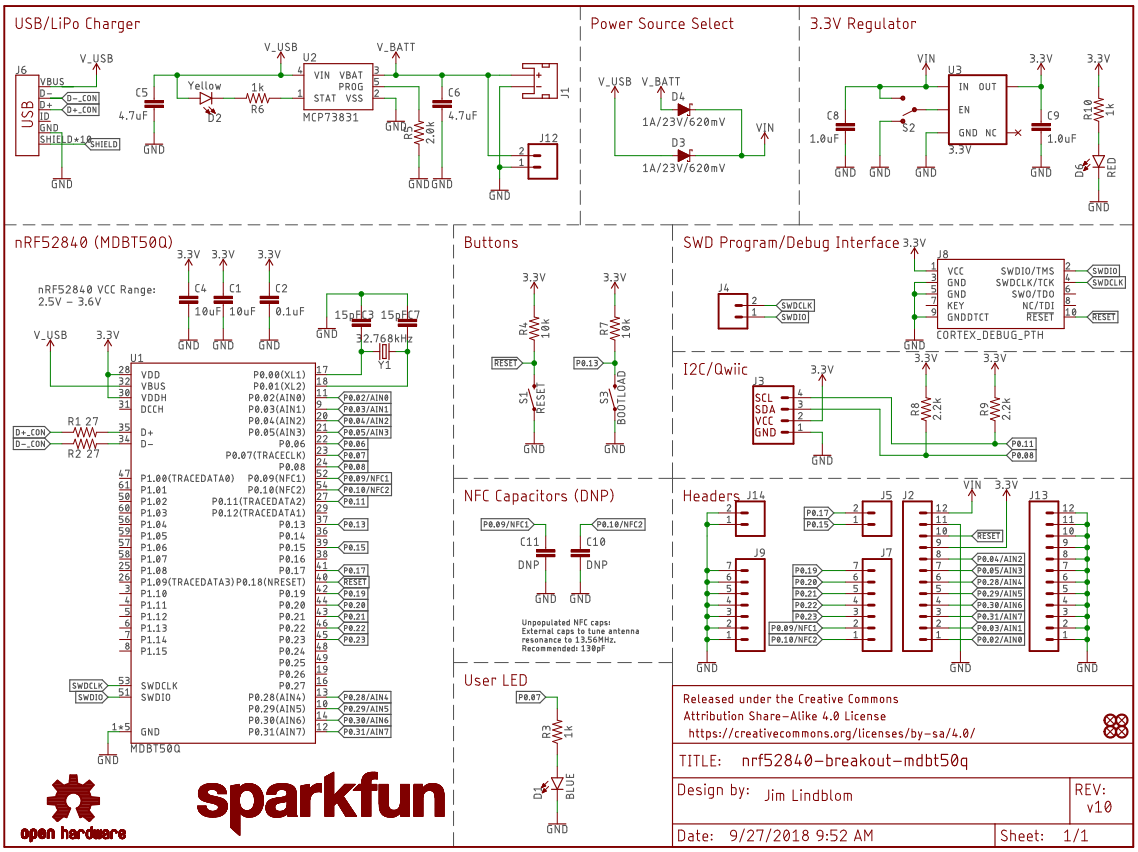


Figure 1: Schematico del microcontrollore nRF52840 [5]

2. **IMU:** nel progetto è stato utilizzato il sensore SparkFun 9DoF IMU Breakout basato sull'ICM-20948 di Invensense[6], un sensore di MotionTracking a 9 assi con consumo energetico estremamente basso, ideale per applicazioni in smartphone, tablet, sensori indossabili e IoT.
   L'ICM-20948 integra un giroscopio a 3 assi, un accelerometro a 3 assi e un magnetometro a 3 assi, racchiuso in un package compatto da 3 mm x 3 mm x 1 mm. Il sensore è posizionato esattamente al centro della scheda, tra i fori di montaggio, per semplificare i calcoli dinamici e ridurre gli errori legati alla posizione del sensore rispetto al sistema fisico. Il sensore supporta un processore digitale di movimento (DMP™) per scaricare la computazione degli algoritmi di motion processing dal processore host, migliorando così le prestazioni energetiche del sistema. Inoltre, è dotato di una calibrazione runtime, che ottimizza automaticamente i parametri del sensore durante l'uso, garantendo misurazioni accurate in tempo reale. Il giroscopio MEMS a 3 assi offre una gamma programmabile di ±250 dps, ±500 dps, ±1000 dps e ±2000 dps, mentre l'accelerometro MEMS a 3 assi può essere configurato con una gamma di ±2g, ±4g, ±8g o ±16g. Il magnetometro ad effetto Hall, anch'esso a 3 assi, ha un intervallo di misurazione fino a ±4900 µT, consentendo di rilevare campi magnetici di varia intensità. Per ottimizzare la gestione dei dati, il sensore è dotato di un FIFO di 512 byte (dimensione che può variare in base alle funzionalità del DMP attive), riducendo la frequenza di comunicazione con il microcontrollore e migliorando l'efficienza complessiva del sistema. La comunicazione tra l'ICM-20948 e il microcontrollore avviene tramite I2C o SPI. L'indirizzo I2C predefinito è 0x69, ma può essere modificato a 0x68 chiudendo un jumper, anche se ciò disabilita la comunicazione SPI. Il sensore supporta l'I2C fino a 400 kHz in modalità rapida, mentre il protocollo SPI può raggiungere velocità fino a 7 MHz, consentendo misurazioni con alta risoluzione temporale. Poiché l'ICM-20948 funziona a 1.8V, la scheda è dotata di traslatori di livello per tutte le linee I/O, il che permette una traduzione bidirezionale del segnale fino a 5.5V. Questo garantisce la compatibilità con una vasta gamma di microcontrollori che operano a tensioni diverse, tra cui 3.3V e 5V. La scheda include anche un regolatore di tensione per facilitare l'alimentazione da 1.8V a 5.5V. Per maggiori informazioni sulle connessioni e sulla configurazione del sensore, si faccia riferimento allo schematico riportato in Figura 2.
   Tra le caratteristiche aggiuntive, vi è un'interfaccia I2C ausiliaria per connettere sensori esterni, conver-

titori ADC a 16 bit integrati, filtri digitali programmabili, un sensore di temperatura e la possibilità di impostare interrupt programmabili. Il dispositivo è compatibile con il software Android e supporta il segnale FSYNC per la stabilizzazione elettronica delle immagini. Per facilitare l'integrazione hardware, la scheda è dotata di due connettori Qwiic, che consentono di collegare facilmente più sensori in cascata. Tuttavia, se preferito, i pin sono disponibili anche con spaziatura standard da 0.1" per l'uso su breadboard. Sono inoltre presenti diversi jumper che permettono di disattivare le resistenze di pull-up del bus I2C, di disattivare il LED di corretta alimentazione e, come gia riportato, cambiare l'indirizzo del dispositivo.

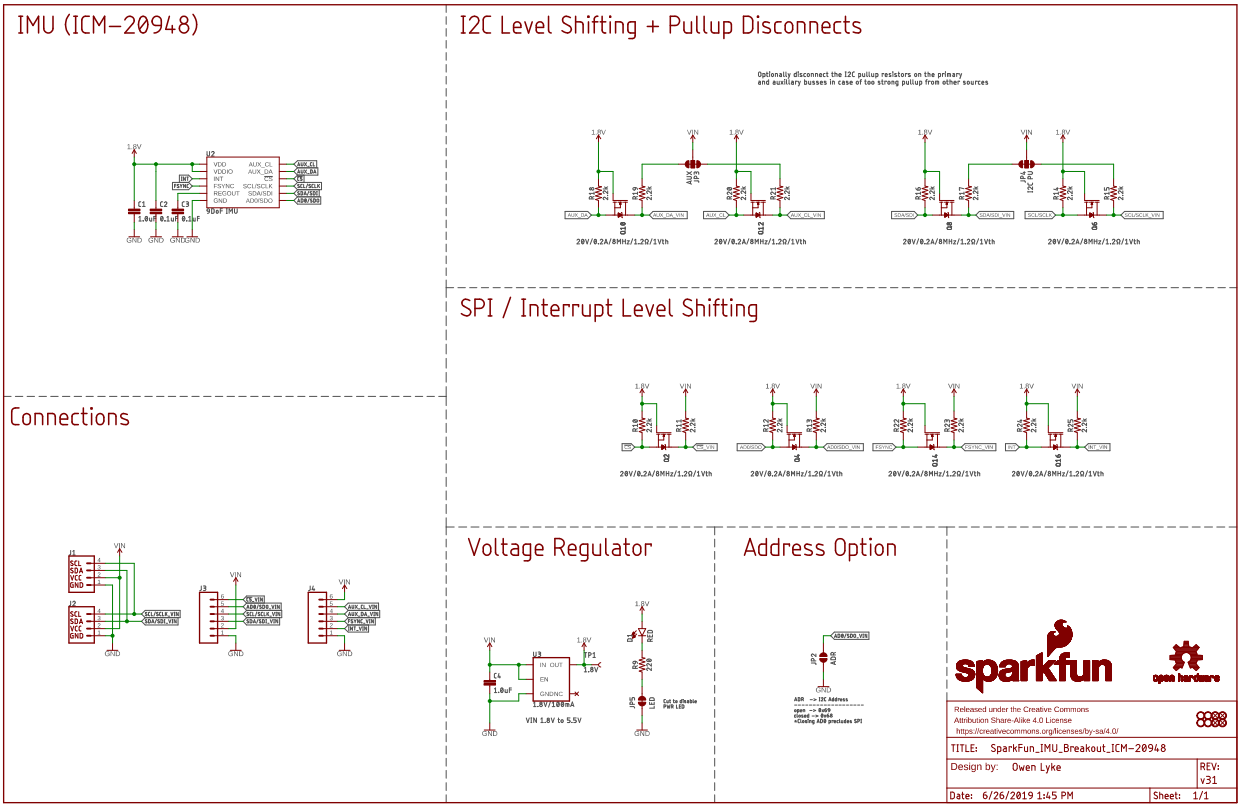


Figure 2: Schematico del sensore SparkFun 9DoF IMU

3. **TOF:** Nel seguente progetto è stato scelto SparkFun Qwiic ToF Imager, un sensore Time-of-Flight (ToF) costruito attorno al VL53L5CX di STMicroelectronics[7].
   Delle due versioni proposte da Sparkfun è stata preferita la versione Mini per la metà dell'ingombro rispetto alla scheda standard Qwiic VL53L5CX. Inoltre, la presenza dei connettori Qwiic sul retro della scheda favorisce il montaggio del sensore rivolto verso l'esterno della struttura del casco, mantenendo i cavi Qwiic all'interno.
   Sfruttando il sistema Qwiic, non è stata necessaria alcuna saldatura per collegarlo al resto del sistema. Tuttavia, sulla scheda del sensore è prevista la possibilità di saldare alcuni header per integrarlo in una breadboard.
   Il chip VL53L5CX proietta un field of view quadrato grazie all'integrazione di elementi ottici diffrattivi, mentre l'obiettivo sul ricevitore concentra il riflesso di luce su una matrice di SPAD.
   E' un sensore multi-zona che può essere impostato per ottenere risoluzioni a 8x8 o 4x4 zone. Le misurazioni della distanza multizona sono possibili fino a 4000 mm, con un ampio campo visivo diagonale di 63°.
   Tra le diverse applicazioni, il sensore Qwiic Mini ToF Imager è ideale per la mappatura di una stanza 3D, il rilevamento di ostacoli ed il riconoscimento dei gesti.
   Facendo riferimento allo schematico riportato in Figura 3, l'alimentazione viene fornita dal connettore Qwiic. In caso contrario, è possibile utilizzare una propria fonte di alimentazione utilizzando i pin che

sono stati disposti lungo il lato inferiore della scheda etichettati come 3V3 e GND.
L'intervallo di tensione che deve essere rispettato in ingresso al sensore è tra 2,7-3,3V.
Allo stesso modo la comunicazione in I2C può avvenire tramite il connettore qwiic o tramite i pin predisposti attraverso i fori SDA e SCL.
Il sensore è dotato di una modalità autonoma a basso consumo programmando il pin di interrupt. Il pin INT è di default configurato come uscita open-drain. Il pin di reset, etichettato come RST, è il pin di reset dell'interfaccia I2C ed è attivo alto: collegato a massa tramite una resistenza da 47 kΩ.
Per l'implementazione di questo progetto, sono stati collegati due sensori TOF attraverso il connettore qwiic in configurazione daisy chain insieme al sensore IMU, descritto nel paragrafo precedente. Per assicurare una corretta connessione, è importante dissaldare le due resistenze di pull-up delle line di comunicazione I2C presenti sulla scheda del sensore. Queste resistenze da 2.2 kΩ sono visibili nello schematico collegate alla rete SDA, SCL. Se non rimosse, insieme alle resistenze di pull-up degli altri sensori, creano un parallelo che riduce la resistenza totale e porta ad un valore logico alto la linea, non permettendo la corretta comunicazione con i sensori.

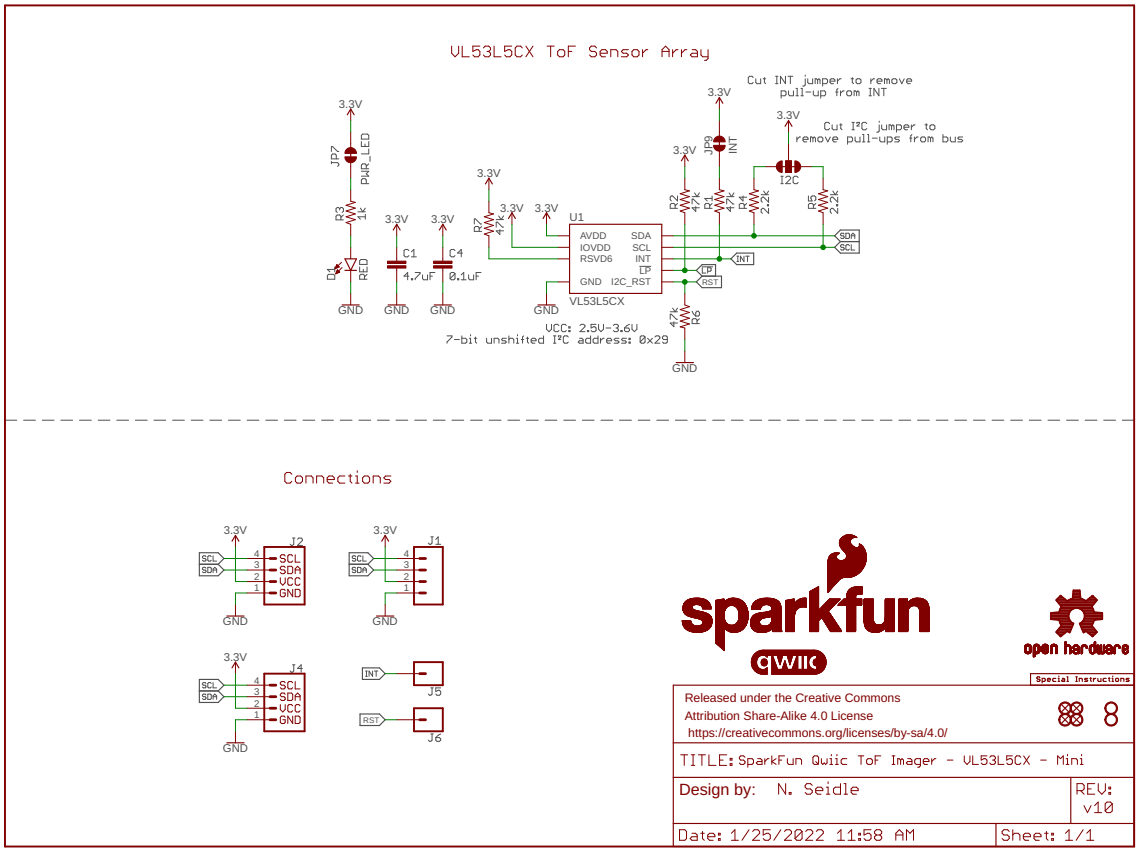


Figure 3: Schematico del sensore SparkFun Mini Qwiic ToF Imager

4. **Batteria LiPo:** La batteria LiPo utilizzata nel seguente progetto appartiene al modello LP504783JU e ha una capacità tipica di 2100 mAh, una tensione nominale di 3.7 V, con un range di tensione di carica che arriva fino a 4.2 V. La batteria può erogare una corrente standard di 420 mA e una corrente massima di 2100 mA, risultando adatta per applicazioni che richiedono sia una carica moderata che una maggiore potenza. La temperatura di lavoro varia da -20°C a +60°C per la scarica e da 0°C a +50°C durante la carica, rendendola utilizzabile in diverse condizioni ambientali. Ha una durata di circa 300 cicli di carica/scarica, mantenendo l'80% della capacità. Inoltre, le sue dimensioni compatte (85mm x 47.5 mm x 5.2 mm) e il peso ridotto di circa 48 grammi, la rendono ideale per l'applicazione. Infatti, grazie alla sua leggerezza e forma snella, può essere facilmente integrata nel casco, senza compromettere il comfort o aggiungere ingombro, un aspetto essenziale per l'usabilità complessiva. Per una maggiore comprensione dell'ingombro si faccia riferimento alla misure riportate in Figura 4.

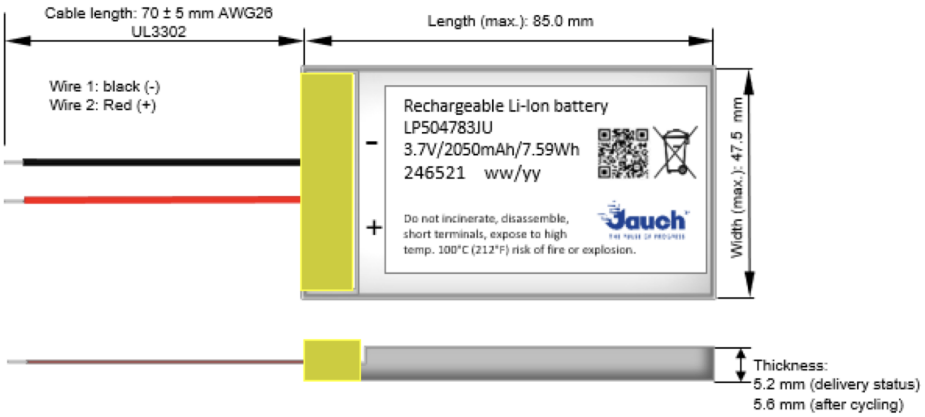


Figure 4: Struttura e dimensioni della Batteria LiPo LP504783JU

5. **Cavetti qwiic:** Il cavetto Qwiic di SparkFun è un cavo a 4 pin progettato per collegare facilmente dispositivi di sviluppo a sensori, display LCD, relè e altri moduli senza necessitare di saldature. Grazie ai connettori JST polarizzati, il cavetto garantisce una connessione corretta, evitando il rischio di scambiare i pin SDA e SCL. Questo semplifica il cablaggio e velocizza la prototipazione, riducendo gli errori di connessione. Inoltre, i cavetti Qwiic permettono di collegare più dispositivi in configurazioni "daisy chain" sulla stessa linea I2C, sfruttando i connettori aggiuntivi presenti su molti moduli. Questo approccio consente di espandere facilmente il sistema, mantenendo la semplicità di cablaggio. Il cavetto è progettato per funzionare con il bus I2C, che non è adatto a comunicazioni su lunghe distanze, e supporta una corrente massima di 226mA. Tuttavia, per estendere la portata del sistema, è possibile utilizzare il Differential I2C Breakout, che consente di trasmettere segnali I2C fino a 30 metri, grazie alla conversione dei segnali in segnali differenziali. I sistemi di SparkFun, come le schede di sviluppo e i moduli, sono progettati per lavorare perfettamente con questi cavetti Qwiic. Molti di questi dispositivi sono dotati di connettori Qwiic già integrati, che facilitano l'interfacciamento diretto con altri moduli Qwiic, senza la necessità di saldature o connessioni complesse.

Di seguito, in Figura 5 e in Figura 6, viene mostrato l'assemblaggio dell'intero hardware descritto, con i vari sensori e componenti collegati, evidenziati dalle frecce per una migliore comprensione.

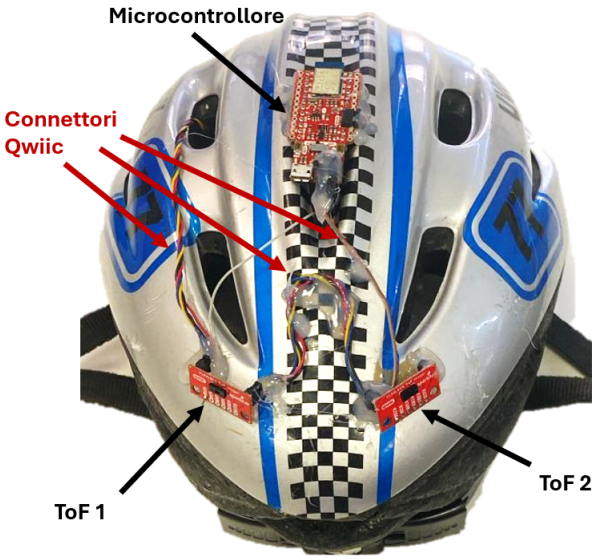


Figure 5: Vista del caschetto dalla parte posteriore. I due sensori ToF sono montati per far sì che i singoli FoV di 63° non si sovrappongono ma si completino.

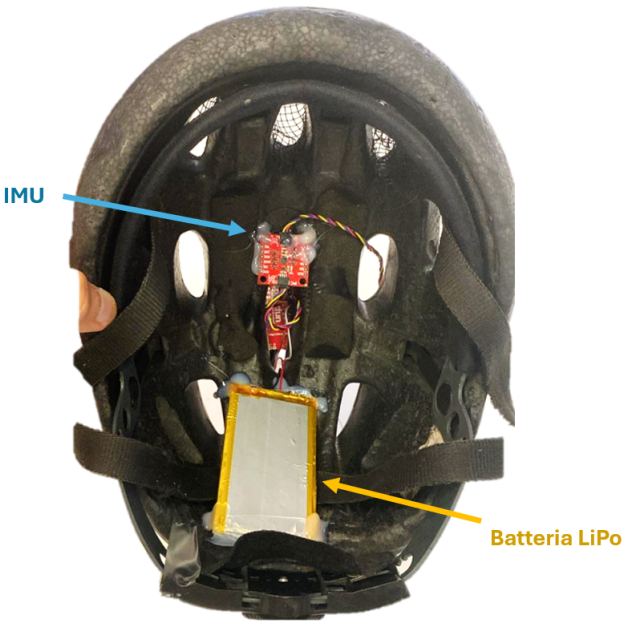


Figure 6: Vista del caschetto nella parte interna

3.1. Progetto PCB

In questo progetto è stato costruito un prototipo di casco sensorizzato che mantiene però una configurazione ingombrante e finalizzata alla sola valutazione dell'implementazione pensata.

In un futuro sviluppo sarebbe opportuno riportare alcuni dei componenti in un circuito stampato per ottenere una maggior compattezza di design e permettere un maggior comfort all'utilizzatore del casco.
Per questo motivo, è stato ideato un circuito stampato tramite il software *Altium*. E' stata ideata una scheda flessibile, per permettere il suo alloggiamento all'interno del telaio del casco. Data l'applicazione del dispositivo e il relativo rischio di caduta, l'inserimento della scheda all'interno del casco non solo ne garantisce l'invisibilità, ma assicura anche una protezione dei componenti interni, in quanto il casco stesso svolge la funzione di involucro protettivo.

Viene di seguito descritta una futura implementazione della scheda all'interno del casco.
La scheda include il microcontrollore, la gestione della batteria ed il sensore IMU. Inoltre, sono previsti alcuni pad liberi che gestiscono i seguenti segnali:

- linea di comunicazione I2C per il segnale di clock
- linea di comunicazione I2C per il canale dati
- il segnale di alimentazione a 3.3V
- il segnale di GND

I pad liberi riportati sostituiscono di fatto il connettore qwiic che viene invece utilizzato nel prototipo. In questo modo, è garantito il collegamento I2C dal microcontrollore ai sensori di prossimità, i quali non sono implementati direttamente sulla scheda; essi dovranno infatti essere esposti all'esterno del casco stesso e collegati ai pad liberi tramite fili esterni.

Nella scheda sono presenti anche altri pad:

- due segnali output denominati RST1 e RST2
- due segnali di output denominati BUZ1,BUZ2

I segnali di reset sono necessari per utilizzare più di un TOF alla volta. Per consentire l'utilizzo simultaneo dei sensori, è necessario modificare il loro indirizzo, operazione che deve essere eseguita configurando i pin di reset di ciascun dispositivo. Per questo motivo è stato pensato di riportare il segnale proveniente dal microcontrollore al pin di reset di ciascun TOF tramite un filo saldato come descritto in precedenza. Sono stati disposti due pad di reset pensando di voler implementare tre TOF e quindi di modificare l'indirizzo solamente a due di essi. La scelta dei tre TOF è stata presa per garantire una simmetricità nel prototipo ipotizzato, ma l'implementazione finale ne vede solo due effettivamente montati. La ragione di ciò è dovuta a difficoltà tecniche riscontrate durante l'assemblaggio.
In una prima bozza del progetto si era pensato di connettere il segnale di allarme per eccessiva vicinanza di altri veicoli a due buzzer acustici. Per questo motivo sono stati progettati i segnali denominati BUZ1 e BUZ2, responsabili di comandare tramite PWM gli attuatori sonori. Anche in questo caso, essendo tali componenti all'esterno del casco, bisogna garantire la loro gestione tramite dei fili saldati. Nel prototipo finale non troveremo questi buzzer in quanto verrà solo inviata una notifica via BLE all'app mobile.
Viene descritto ora più nel dettaglio il progetto della scheda.
I componenti presenti sul circuito stampato sono i seguenti:

microcontrollore	NRF-52840 (MDBT50Q)
connettore batteria	S2B-PH-K
connettore USB	105133-0001
caricatore batteria LiPo	MCP73831-2DCI/MC
regolatore di tensione	LD39020ADTPU33R
sensore IMU	ICM-20948
componenti passivi	resistori e capacità
singoli pad	

In Figura 7 viene riportato lo schematico contenente il microcontrollore e la gestione dell'alimentazione.
Per quanto riguarda l'alimentazione della scheda, essa proviene dalla batteria LiPo che fornisce 3.7V tramite il connettore J2. Questa tensione, denominata *BATTpos*, viene gestita da un regolatore di tensione, *U3*, che restituisce i 3.3V necessari ad alimentare il microcontrollore ed i sensori. Il componente *U2* gestisce la ricarica della batteria se la scheda viene collegata tramite il connettore *USB J1*.
Il **microcontrollore**, nello schematico denominato *U1*, è il medesimo utilizzato nel prototipo, ad eccezione del package: nel prototipo utilizziamo la versione di Sparkfun, mentre nella scheda è prevista la sua versione implementabile su circuito stampato (MDBT50Q). Il microcontrollore è alimentato attraverso i pin di alimentazione

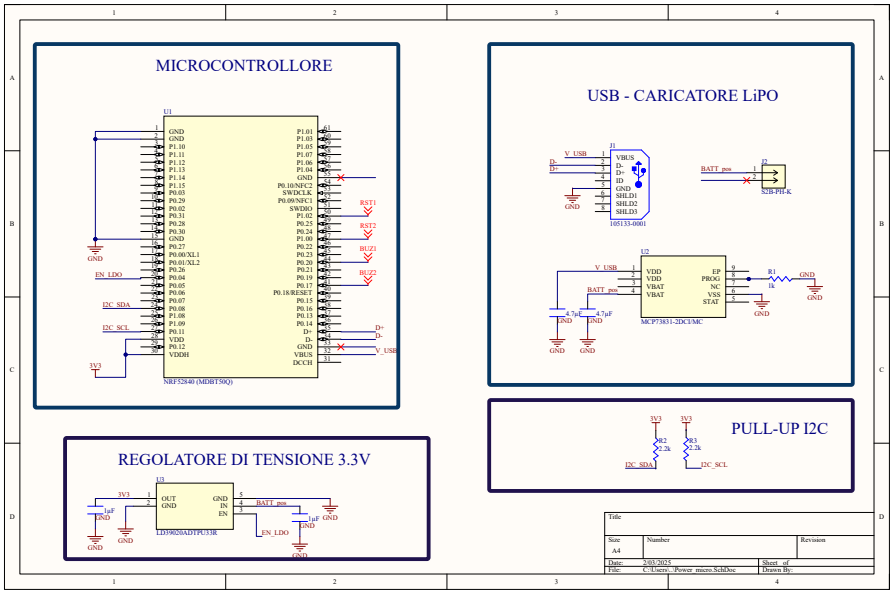


Figure 7: Schematico microcontrollore e gestione alimentazione

VDD e *VDDH* i quali sono connessi alla tensione 3V3 precedentemente descritta. I pin *D+*, *D-* e *VBUS* sono connessi ai pin *D+*, *D-* e *VUSB* del connettore USB. Per utilizzare la comunicazione I2C, il pin *P0.08* gestisce la linea dati, mentre il pin *P0.11* la linea di clock. Sono presenti inoltre due resistenze di pull-up per tali segnali, collegate all'alimentazione 3V3: le resistenze R2, R3. Infine, per i segnali di controllo dei buzzer e di reset dei TOF sono stati collegati i pin *P1.02, P1.00, P0.20, P0.17*. Questi ultimi nello schematico sono segnalati come off-sheet connector in quanto sono collegati ai pad singoli riportati in un secondo foglio di schematico per maggior chiarezza.
In Figura 8 viene riportato lo schematico contenente il sensore IMU e i pad singoli.

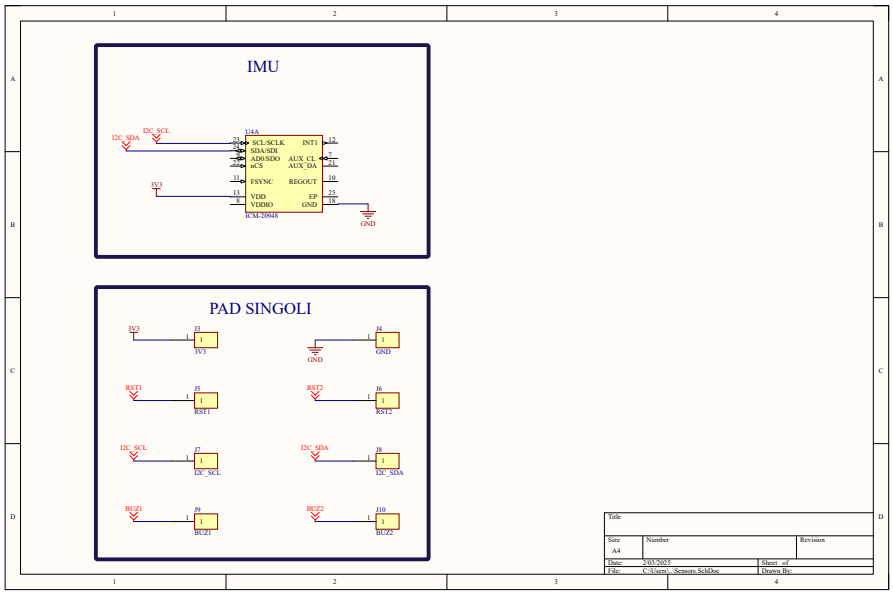


Figure 8: Schematico IMU e pad singoli

Per quanto riguarda il **sensore IMU**, sono stati abilitati i pin di alimentazione (*PIN13* connesso a 3V3) e ground (*PIN18*). Inoltre, i pin 23 e 24 sono connessi rispettivamente al segnale di clock e di dati della comunicazione I2C. L'indirizzo del sensore IMU è 110100X , in cui l'ultimo bit è dato dal livello logico del pin *AD0*. Nello schematico tale pin andrebbe collegato all'alimentazione o a massa a seconda di come si voglia impostare l'indirizzo. Tale configurazione permetterebbe di avere contemporaneamente due sensori IMU connessi allo

stesso canale di comunicazione ma anche di cambiare l'indirizzo qual ora corrispondesse a uno degli indirizzi di altri sensori.
I pad singoli riportati nello schematico sono necessari a per gestire i segnali di alimentazione, comunicazione I2C, segnali di reset e gestione buzzer.
In Figura 9 viene riportata una visione tridimensionale del circuito stampato.

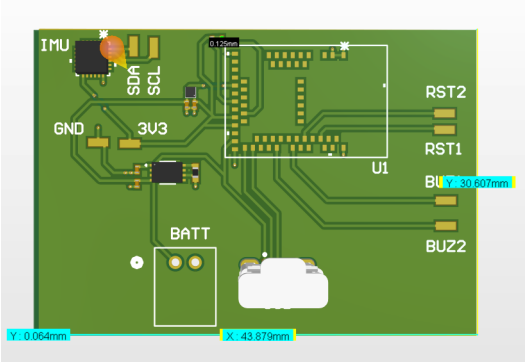


Figure 9: Circuito stampato 3D con quotature

I componenti sono stati disposti in modo tale da minimizzare le dimensioni. Sono riportate in figura le misure della scheda in millimetri ottenute tramite il software *Altium* che permette di quotare il design del circuito stampato. Inoltre, la posizione dei singoli pad è stata scelta in modo tale da essere accessibili alla saldatura dei fili e non compromettere la funzionalità di altri componenti. Per questo motivo sono posizionati verso l'esterno della scheda. Inoltre, sono state predisposte alcune scritte da stampare in serigrafia, utili per la saldatura dei fili e per il riconoscimento dei vari componenti.
Infine, per quanto riguarda il layout, la scheda è stata pensata come composta da due soli strati. Uno strato superficiale, denominato TOP, in cui sono montati i componenti e in cui si trovano le piste dei diversi segnali. Con lo strumento poligono è stato creato un piano di alimentazione su questo strato, che riporta su tutta la scheda i 3.3V. Uno strato sottostante, il BOTTOM, utilizzato per il piano di massa della scheda. Nella Figura 10, si riporta una visione del layout comprendente entrambi gli strati; il colore rosso si riferisce allo strato TOP mentre il colore blu allo strato BOTTOM, mentre le scritte in serigrafia appartengono ad uno strato denominato Overlay.

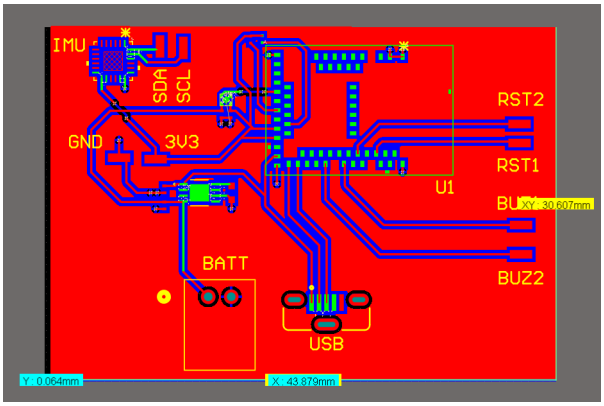


Figure 10: Layout circuito stampato

4. Firmware

Il seguente firmware ha l'obiettivo di gestire la raccolta, l'elaborazione e la trasmissione di dati provenienti dai diversi sensori. In particolare vengono gestiti una IMU, per la rilevazione delle cadute e l'algoritmo di attention, e 2 sensori ToF per la rilevazione dei veicoli/ostacoli in avvicinamento. Inoltre, la gestione del BLE permette la comunicazione con dispositivi esterni.
Lo schema a blocchi in Figura 11 rappresenta la struttura generale del firmware, suddivisa in più parti, ciascuna delle quali verrà approfondita nei paragrafi successivi. Le principali fasi includono l'inizializzazione e la calibrazione dei sensori, l'applicazione di algoritmi per la gestione e filtraggio dei dati (filtro di Kalman, algoritmo

di fall detection, algoritmo di attention, algoritmo TOF), ed infine la trasmissione dei risultati ottenuti.
L'interazione tra queste componenti permette al sistema di monitorare in tempo reale le condizioni del ciclista e di rilevare eventuali cadute o anomalie in modo affidabile ed efficace.
Tutto il firmware è implementato utilizzando Arduino IDE.

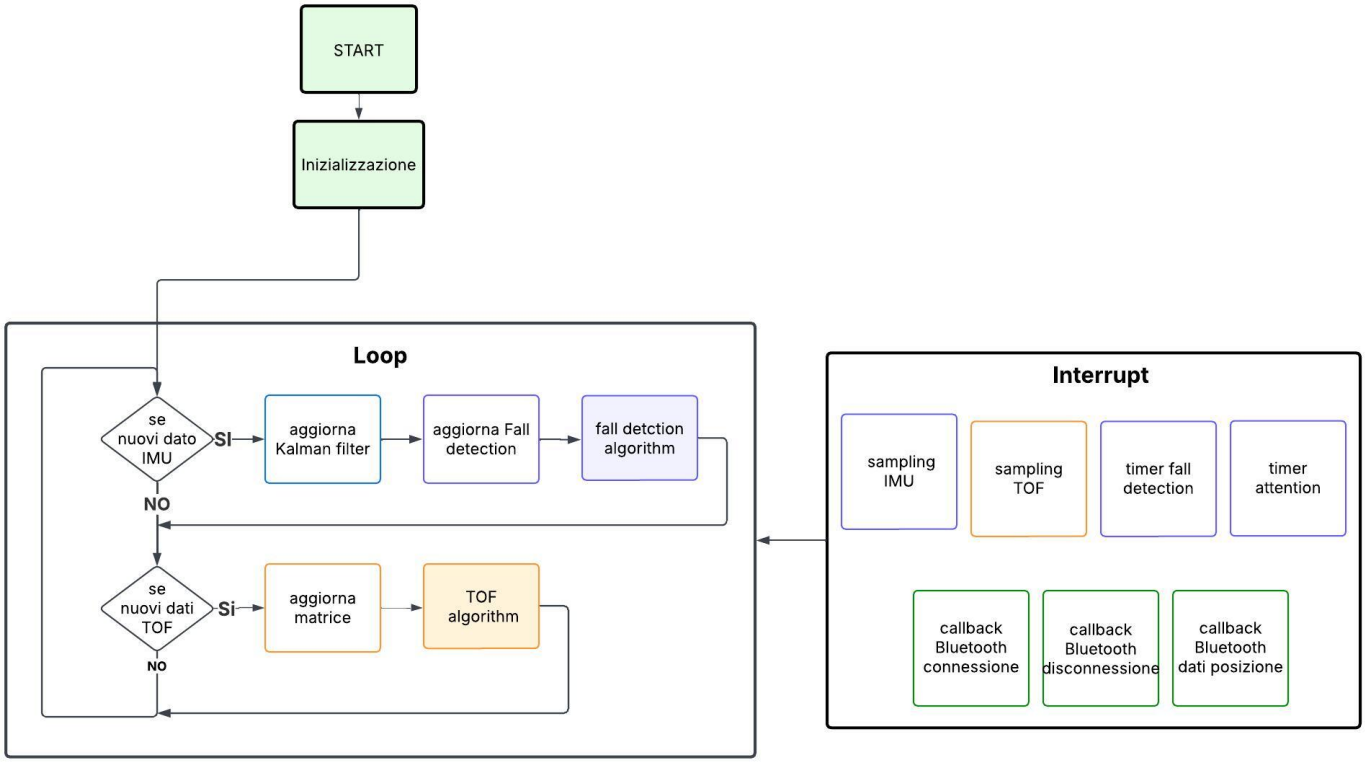


Figure 11: Schema a blocchi generale. In azzurro i blocchi riguardanti il Kalman Filter, in viola quelli per la fall detection, in arancione la misurazione della distanza ed in verde quelli per la comunicazione BLE

4.1. Calibrazione

4.1.1 Calibrazione Accelerometro

La calibrazione dell'accelerometro è stata effettuata per ridurre gli errori sistematici legati al bias e migliorare l'accuratezza delle misurazioni. Il processo ha previsto la determinazione dell'offset e del fattore di scala attraverso una regressione lineare, sfruttando la natura costante dell'accelerazione gravitazionale terrestre (1 g). Per ottenere i dati necessari, l'IMU è stata posizionata in tre diverse configurazioni statiche: con l'asse z orientato verso l'alto, con l'asse x verso l'alto e con l'asse y verso l'alto. Queste misurazioni hanno permesso di stimare l'offset dell'accelerometro, dato che in condizioni ideali ogni asse, quando perfettamente allineato con la gravità, dovrebbe registrare un'accelerazione di ± 1 g. Per determinare il fattore di scala, sono state effettuate ulteriori misurazioni posizionando il sensore con ciascun asse orientato sia verso l'alto (+1 g) sia verso il basso (-1 g). In assenza di dispositivi meccanici in grado di applicare accelerazioni note lungo i tre assi, il fattore di scala è stato calcolato sfruttando la differenza tra le misure di accelerazione positive e negative per un medesimo asse, secondo la relazione:

$$m = \frac{2}{\text{mean}(z_+) - \text{mean}(z_-)} \tag{1}$$

dove $\text{mean}(z_+)$ e $\text{mean}(z_-)$ rappresentano le medie delle misurazioni acquisite quando l'asse z era orientato rispettivamente verso l'alto e verso il basso. Lo stesso metodo è stato applicato agli assi x e y. La raccolta dei dati è stata effettuata utilizzando il software CoolTerm, che ha permesso di generare file di testo contenenti le misurazioni per ogni configurazione dell'IMU. Questi file sono stati successivamente convertiti in formato CSV e analizzati tramite uno script Python, che ha calcolato automaticamente i valori medi e determinato i parametri di calibrazione. Questo processo, eseguito una sola volta per ogni sensore, ha garantito una maggiore

affidabilità nelle misurazioni successive, riducendo gli errori dovuti alle imperfezioni di fabbricazione e alle variazioni di sensibilità tra i diversi assi.

4.1.2 Calibrazione del Giroscopio

La calibrazione del giroscopio è stata effettuata, come per l'accelerometro, per ridurre l'errore sistematico dovuto al bias e migliorare l'accuratezza delle misurazioni, un aspetto particolarmente critico in un sistema di navigazione inerziale, dove il giroscopio è soggetto a fenomeni di drift che possono compromettere la precisione della stima dell'orientazione. Per eseguire la calibrazione, il fattore di scala m è stato determinato con lo stesso metodo utilizzato per l'accelerometro, confrontando le misurazioni ottenute nelle rotazioni controllate, in assenza di un sistema meccanico in grado di generare velocità angolari note. L'IMU è stata quindi inserita in un case che ne garantiva il corretto alloggiamento e allineamento. Seguendo il metodo suggerito dal Prof. Langfelder, in assenza di piastre rotanti da laboratorio, il sensore è stato ruotato manualmente in modo controllato. La procedura ha previsto inizialmente il posizionamento dell'IMU con l'asse z orientato verso l'alto. Successivamente, il case è stato ruotato di 90° fino a portare l'asse z in una direzione parallela al piano. Dopo aver raggiunto questa posizione, l'operazione è stata ripetuta in senso opposto, riportando l'asse z alla configurazione iniziale perpendicolare al piano. Il movimento da 0° a 90° è stato considerato sufficientemente *smooth*, permettendo di calcolare la velocità angolare istantanea derivando la posizione angolare. La calibrazione è stata effettuata su tutti e tre gli assi (pitch, roll e yaw) registrando i dati di velocità angolare in °/s (DPS) durante il passaggio da 0° a 90°. Integrando i valori medi della velocità angolare, il risultato atteso era di 90°, verificando così la correttezza della misura. Per determinare l'offset del giroscopio, l'IMU è stata mantenuta completamente ferma, registrando la velocità angolare in condizioni statiche a *0 DPS*. Tale offset è stato poi sottratto dai dati raccolti durante il normale utilizzo del sensore. La calibrazione del giroscopio si è rivelata essenziale per garantire misure affidabili nel contesto della navigazione inerziale, poiché il drift accumulato nel tempo può portare a errori significativi nella stima dell'orientazione del sistema.

4.1.3 Calibrazione del Magnetometro

I magnetometri sono strumenti fondamentali per misurare l'intensità e la direzione del campo magnetico terrestre, spesso utilizzati in sistemi di navigazione per determinare l'orientamento. Tuttavia, le misurazioni possono essere influenzate da distorsioni magnetiche causate da materiali ferromagnetici presenti nelle vicinanze del sensore. Queste distorsioni si classificano in due categorie principali:

- **Distorsioni Hard Iron**: sono causate da sorgenti magnetiche permanenti, come magneti o componenti elettrici, che generano un campo magnetico costante e aggiungono un offset fisso alle misurazioni del magnetometro. Queste distorsioni spostano il centro della sfera dei dati magnetici lontano dall'origine.
- **Distorsioni Soft Iron**: derivano da materiali che deformano il campo magnetico terrestre senza generare un proprio campo magnetico permanente. Queste distorsioni alterano la forma della sfera dei dati magnetici, trasformandola in un'ellissoide.

Per ottenere misurazioni accurate, è essenziale calibrare il magnetometro compensando entrambe le distorsioni. Una tecnica comune consiste nel ruotare il sensore in tutte le direzioni, raccogliendo dati che rappresentano una sfera ideale in assenza di distorsioni. Successivamente, si applicano algoritmi matematici per correggere gli effetti di hard e soft iron, riportando i dati a una sfera centrata nell'origine. Per la calibrazione del nostro magnetometro, abbiamo utilizzato **MotionCal**, un programma sviluppato da Paul Stoffregen. MotionCal è uno strumento cross-platform che facilita la calibrazione sia per le distorsioni hard iron che soft iron. Fornisce una visualizzazione 3D dei dati del magnetometro e consente di identificare e scartare valori anomali, oltre a valutare la copertura sferica dei dati raccolti [8]. Il processo di calibrazione con MotionCal prevede la raccolta di dati mentre il sensore viene ruotato lentamente in tutte le direzioni, assicurando una copertura completa della sfera. Il software analizza i dati, visualizzando in tempo reale la distribuzione dei punti nello spazio tridimensionale. Una volta raccolti dati sufficienti, MotionCal calcola i parametri di calibrazione necessari per compensare le distorsioni hard e soft iron, migliorando così l'accuratezza delle misurazioni del magnetometro.

4.2. Kalman filter

Quando si lavora con sensori come giroscopi, accelerometri e magnetometri per stimare l'orientamento di un oggetto nello spazio, il problema principale è il rumore nelle misure. Ogni sensore ha limiti fisici e introduce errori, dovuti a disturbi esterni, variazioni ambientali o imperfezioni nella calibrazione. Il giroscopio, ad esempio, fornisce informazioni sulla velocità angolare, tuttavia, se lo usassimo da solo per stimare l'orientamento, attraverso l'integrazione delle velocità nel tempo, gli errori si accumulerebbero rapidamente, portando a una deriva della stima dell'orientamento del corpo nel tempo. Gli accelerometri e i magnetometri possono correggere

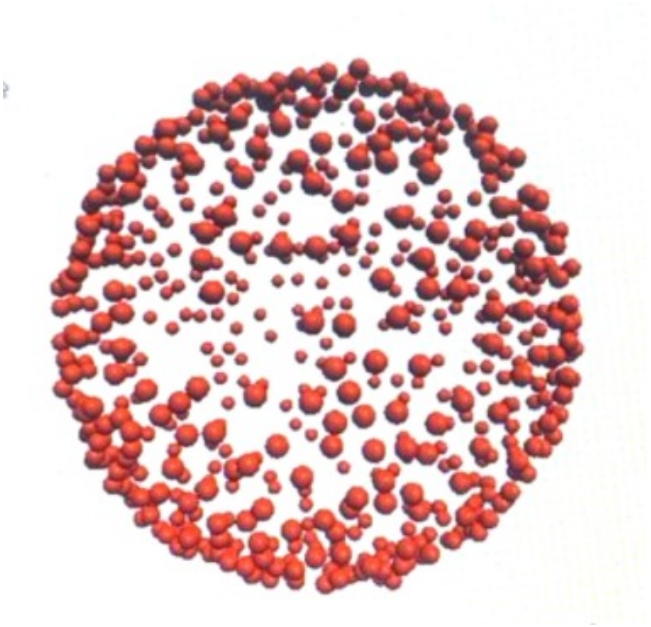


Figure 12: Schermata finale dopo la calibrazione del magnetometro. Ogni punto si riferisce ad un dato acquisito.

questa deriva, ma anch'essi soffrono di rumore e di interferenze esterne. La soluzione ottimale, pertanto, è una "Sensor Fusion", ovvero l'integrazione di misura provenienti da più sensori per ottenere una rappresentazione dello stato con una accuratezza migliore rispetto ad una ottenuta se si utilizzassero i sensori separati.
Il Kalman Filter è un metodo matematico che combina tutte queste misure in modo ottimale, stimando lo stato del sistema (in questo caso l'orientazione) riducendo l'incertezza. Questo avviene grazie a due fasi: una predizione basata su un modello dinamico e una correzione che utilizza nuove osservazioni per aggiornare la stima. Quando il modello non è lineare, come nel caso dell'orientazione con quaternioni, si utilizza la versione estesa del filtro, l'Extended Kalman Filter (EKF), che approssima il sistema usando la sua derivata (Jacobiano).

4.2.1 Il Gimbal lock

L'orientamento di un corpo rigido nello spazio può essere rappresentata con angoli di Eulero o con matrici di rotazione. Gli angoli di Eulero sono intuitivi, ma hanno un grosso problema: il gimbal lock. Questo accade quando due assi di rotazione si allineano, causando la perdita di un grado di libertà.
Un esempio pratico è il movimento di un aereo, che si orienta nello spazio attraverso tre angoli di Eulero:

- Roll: rotazione attorno all'asse longitudinale dell'aereo.
- Pitch: rotazione attorno all'asse trasversale.
- Yaw: rotazione attorno all'asse verticale.

Se l'aereo alza il muso fino a puntare perfettamente verso l'alto (pitch = 90°), gli assi di roll e yaw si allineano. Questo significa che una rotazione attorno a uno dei due avrà lo stesso effetto dell'altro, rendendo impossibile un controllo completo dell'orientazione senza cambiare il pitch.
Le matrici di rotazione evitano questo problema, ma occupano più memoria e richiedono più operazioni nei calcoli.
Grazie ai quaternioni, il problema del gimbal lock viene risolto e i calcoli semplificati, con lo svantaggio di essere di difficile interpretazione in quanto numeri complessi.

4.2.2 Quaternioni

I quaternioni sono una rappresentazione matematica delle rotazioni nello spazio tridimensionale e possono essere espressi come una combinazione di una parte scalare e un vettore tridimensionale. Un quaternione $q = (q_0, q_1, q_2, q_3)$ che rappresenta una rotazione di un angolo θ attorno a un asse unitario $\mathbf{u} = (u_x, u_y, u_z)$ è dato dalla seguente espressione:

$$q = (q_0, q_1, q_2, q_3) = \left(\cos\frac{\theta}{2}, u_x \sin\frac{\theta}{2}, u_y \sin\frac{\theta}{2}, u_z \sin\frac{\theta}{2}\right)$$

Rispetto agli angoli di Eulero, i quaternioni presentano il vantaggio di evitare il problema della singolarità (gimbal lock) e di essere più efficienti nelle operazioni di composizione delle rotazioni, poiché non richiedono conversioni tra matrici di rotazione o angoli intermedi. Essendo su un microcontrollore con risorse computazionali limitate, la scelta dei quaternioni viene ulteriormente preferita.

4.2.3 Etended Kalman Filter (EKF)

L' *Extended Kalman Filter* (EKF) è un algoritmo ampiamente utilizzato per la stima dello stato di un sistema dinamico di un sistema non lineare che presenta rumore nelle misure. In questo contesto, l'EKF viene impiegato per determinare l'orientamento di un corpo rigido nello spazio, rappresentato attraverso quaternioni, utilizzando le osservazioni fornite da giroscopi, accelerometri e magnetometri. La differenza principale tra il Kalman filter (KF) e l'EKF sta nel fatto che quest'ultimo linearizza un sistema non lineare.
La stima dello stato si basa sulla combinazione di due fonti di informazione:

1. **Il modello del sistema**, che descrive l'evoluzione dinamica dell'orientamento nel tempo attraverso equazioni differenziali.
2. **Le misure dei sensori**, che forniscono osservazioni dirette ma affette da rumore e imprecisioni.

Un sistema dinamico è caratterizzato da uno stato che varia nel tempo, e la sua evoluzione può essere modellata matematicamente. Tuttavia, oltre al rumore dei sensori, esiste anche un'incertezza intrinseca nel modello stesso, dovuta a semplificazioni adottate nella descrizione del sistema (linearizzazione). Per questo motivo, la stima dell'orientamento non può essere deterministica, ma deve essere continuamente corretta sulla base delle misure acquisite.
L'EKF affronta questo problema aggiornando iterativamente la stima dell'orientamento: in una prima fase utilizza il modello dinamico per prevedere l'evoluzione dello stato, mentre in una seconda fase integra le misure dei sensori per correggere tale previsione. Il processo si suddivide in due fasi principali:

- **Prediction step**: stima il prossimo stato e la sua covarianza a partire dallo stato precedente.
- **Correction step**: aggiorna la stima incorporando nuove misurazioni per ridurre l'incertezza.

4.2.4 Prediction step

$$\hat{\mathbf{x}}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t \tag{2}$$

$$\hat{\mathbf{P}}_t = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^T + \mathbf{Q}_t \tag{3}$$

4.2.5 Correction step

$$\mathbf{v}_t = \mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_t \tag{4}$$

$$\mathbf{S}_t = \mathbf{H}\hat{\mathbf{P}}_t\mathbf{H}^T + \mathbf{R} \tag{5}$$

$$\mathbf{K}_t = \hat{\mathbf{P}}_t\mathbf{H}^T\mathbf{S}_t^{-1} \tag{6}$$

$$\mathbf{x}_t = \hat{\mathbf{x}}_t + \mathbf{K}_t\mathbf{v}_t \tag{7}$$

$$\mathbf{P}_t = \hat{\mathbf{P}}_t - \mathbf{K}_t\mathbf{S}_t\mathbf{K}_t^T \tag{8}$$

- $\mathbf{x}_t \in R^n$: stato stimato del sistema al tempo t.
- $\hat{\mathbf{x}}_t \in R^n$: stato previsto prima della correzione.
- $\mathbf{P}_t \in R^{n\times n}$: covarianza dell'errore di stima dopo l'aggiornamento con le misure.
- $\hat{\mathbf{P}}_t \in R^{n\times n}$: covarianza dell'errore di stima prima della correzione.
- $\mathbf{F} \in R^{n\times n}$: matrice di transizione dello stato.
- $\mathbf{B} \in R^{n\times k}$: modello di ingresso di controllo.
- $\mathbf{u}_t \in R^k$: vettore di input di controllo.
- $\mathbf{Q}_t \in R^{n\times n}$: covarianza del rumore del processo.
- $\mathbf{z}_t \in R^m$: misura osservata al tempo t.
- $\mathbf{H} \in R^{m\times n}$: matrice del modello di osservazione.
- $\mathbf{v}_t \in R^m$: residuo dell'innovazione o errore di misura.
- $\mathbf{S}_t \in R^{m\times m}$: covarianza della predizione della misura.
- $\mathbf{R} \in R^{m\times m}$: covarianza del rumore della misura.
- $\mathbf{K}_t \in R^{n\times m}$: guadagno di Kalman, che determina l'aggiornamento dello stato basato sulle misure.

Queste equazioni descrivono il processo iterativo del filtro: prima viene eseguita la **predizione** del nuovo stato e della sua incertezza, successivamente si applica la **correzione** sulla base delle misure disponibili per ottenere una stima più precisa.
Nel caso dell'Extended Kalman Filter, la predizione dello stato avviene attraverso una funzione non lineare $f(\mathbf{x}_{t-1}, \mathbf{u}_t)$ e il modello di osservazione è anch'esso non lineare, descritto da $h(\mathbf{x}_t)$. Per applicare il filtro, entrambe le funzioni vengono linearizzate tramite lo sviluppo di Taylor al primo ordine, utilizzando le matrici Jacobiane **F** e **H**.

4.2.6 Prediction phase

I dati del giroscopio vengono trattati come input esterni al filtro e non come misure dirette. In altre parole il giroscopio fornisce l'informazione sull'angolo di rotazione ma il filtro considera i suoi dati come parte del modello, piuttosto che una misura diretta dell'orientamento. Questo implica che le misure che correggono lo stato del sistema sono quelle fornite dall'accelerometro e dal magnetometro. La fase di predizione calcola l'orientamento al tempo t partendo dalla stima dell'orientamento al tempo t-1 e integrando le velocità angolari fornite dal giroscopio (con la formula di **Euler-Rodrigues**). Il rumore proviene principalmente dai giroscopi e per questo la sua accuratezza influisce direttamente sulla qualità della stima dell'orientamento. La matrice di covarianza del rumore **Qt** descrive come la varianza del giroscopio influisce sulla predizione. In generale si assume che i rumori delle diverse componenti del giroscopio siano non correlati, semplificando così il modello. Per calcolare l'evoluzione della covarianza dell'errore di stato durante la predizione viene utilizzato il Jacobiano delle misurazioni del giroscopio, derivando la F non più rispetto al quaternione, ma rispetto alla velocità angolare. Nel contesto dell'Extended Kalman Filter (EKF), il quaternione q rappresenta lo stato del sistema e descrive l'orientamento del sensore rispetto al sistema globale. Il giroscopio fornisce le velocità angolari $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]$ nel sistema locale del sensore.

$$\dot{q} = \frac{1}{2} q \otimes \Omega, \tag{9}$$

dove $\Omega = [0, \omega_x, \omega_y, \omega_z]$ è il quaternione esteso delle velocità angolari e $\otimes$ indica il prodotto tra quaternioni.
L'integrazione numerica consente di aggiornare il quaternione:

$$q_{k+1} = q_k + \dot{q}\Delta t, \tag{10}$$

seguendo di una normalizzazione per preservare la sua unitarietà:

$$q_{k+1} \leftarrow \frac{q_{k+1}}{\|q_{k+1}\|} \tag{11}$$

Questa formulazione permette di combinare direttamente le misurazioni del giroscopio, evitando trasformazioni esplicite tra sistemi di riferimento, garantendo un aggiornamento coerente dell'orientamento. Durante questa fase, la matrice P viene aggiornata utilizzando le matrici Q ed F. Occorre identificare delle condizioni al contorno al fine di una corretta convergenza del filtro:

- P deve essere simmetrica e definita positiva (SDP): questo è fondamentale per non creare instabilità numerica (autovalori della matrice P negativi) o propagarsi nel processo di correction e rendere singolare la matrice S;
- Durante la fase di stima la matrice di covarianza dell'errore P deve aumentare: ciò fa sì che la confidenza del modello diminuisca, evitando che la stima del quaternione sia preponderante sulla correzione;

4.2.7 Correction phase

Il correction step del Filtro di Kalman Esteso (EKF) serve a correggere la stima dell'orientamento ottenuta nella fase di predizione, utilizzando le misurazioni provenienti da accelerometro e magnetometro. Questi sensori forniscono informazioni sull'orientamento del dispositivo rispetto al frame globale che può essere il NED (asse X del sistema di riferimento relativo allineato col Nord geografico, asse Y a Est e asse Z diretto come l'accelerazione di gravità) o ENU (assi X, Y, Z orientati rispettivamente a Est, Nord e opposto all' accelerazione di gravità), mentre il giroscopio fornisce misurazioni nel frame del sensore, che è il sistema di coordinate locale del dispositivo. Questo comporta la necessità di ruotare le letture accelerometriche e magnetometriche per allinearle con la previsione dell'orientamento nel frame del sensore, ottenuta dal giroscopio.
Il primo passo consiste nell'allineamento delle misure delle intensità di accelerazione di gravità e di campo magnetico "teoriche" $\mathbf{z} = [\mathbf{a}; \mathbf{m}]$, (ovvero quelle che si avrebbero se la terna mobile relativa della IMU fosse perfettamente allineata con il sistema di riferimento NED o ENU). La rotazione dei vettori accelerazione **a** e campo magnetico **m** usando la matrice di rotazione derivata dal quaternione $\hat{q}_t$ è data da:

$$\mathbf{a}' = \mathbf{C}(\hat{q}_t)\mathbf{a} \tag{12}$$

$$\mathbf{m}' = \mathbf{C}(\hat{q}_t)\mathbf{m} \tag{13}$$

dove la matrice di rotazione $\mathbf{C}(\hat{q}_t)$ è:

$$\mathbf{C}(\hat{q}_t) = \begin{bmatrix} \hat{q}_w^2 + \hat{q}_x^2 - \hat{q}_y^2 - \hat{q}_z^2 & 2(\hat{q}_x\hat{q}_y - \hat{q}_w\hat{q}_z) & 2(\hat{q}_x\hat{q}_z + \hat{q}_w\hat{q}_y) \\ 2(\hat{q}_x\hat{q}_y + \hat{q}_w\hat{q}_z) & \hat{q}_w^2 - \hat{q}_x^2 + \hat{q}_y^2 - \hat{q}_z^2 & 2(\hat{q}_y\hat{q}_z - \hat{q}_w\hat{q}_x) \\ 2(\hat{q}_x\hat{q}_z - \hat{q}_w\hat{q}_y) & 2(\hat{q}_y\hat{q}_z + \hat{q}_w\hat{q}_x) & \hat{q}_w^2 - \hat{q}_x^2 - \hat{q}_y^2 + \hat{q}_z^2 \end{bmatrix} \tag{14}$$

Quindi, applicandola ai due vettori:

$$\begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} = \mathbf{C}(\hat{q}_t) \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \tag{15}$$

$$\begin{bmatrix} m'_1 \\ m'_2 \\ m'_3 \end{bmatrix} = \mathbf{C}(\hat{q}_t) \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix} \tag{16}$$

Queste misure rappresentano le misure di campo magnetico e di accelerazione che il sensore dovrebbe misurare se la stima della predizione fosse corretta al 100%. Una volta che le misure sono trasformate nel giusto sistema di riferimento, si applica la **"innovation"**, che è la differenza tra la misura attuale della IMU e quella prevista allo step precedente. A questo punto, entra in gioco il **Kalman Gain**, che è un valore di bilanciamento che decide quanto della differenza tra la misura attuale e quella prevista (innovazione) deve essere considerata per aggiornare lo stato stimato. Se il Kalman Gain è pari a 1, l'aggiornamento avviene completamente in base alla nuova misura, mentre se è pari a 0, non viene fatto alcun aggiornamento e lo stato rimane invariato.
Per il calcolo della misura prevista per l'accelerometro e il magnetometro, si utilizzano le direzioni di gravità e del campo magnetico, che vengono stimate nel frame globale e poi ruotate nel frame del sensore. La misura dell'accelerometro è il vettore di gravità, mentre la misura del magnetometro è il vettore del campo magnetico. Entrambi i vettori vengono normalizzati per considerare solo le loro direzioni e non il modulo.
La matrice **S** (5) è una parte fondamentale del processo di correzione nell'EKF. Essa rappresenta la covarianza delle misure di innovazione, che è il grado di incertezza associato alla differenza tra la misura attuale e la misura prevista. In altre parole, la matrice S aiuta a descrivere quanto la misura innovativa (la differenza tra la previsione e la misurazione) è affidabile. Per il calcolo di S si considera R, ossia la matrice di covarianza dei rumori di processo di accelerometro e magnetometro, che viene semplificata ipotizzando i rumori come indipendenti.
Un punto cruciale per il buon svilupo dell'algoritmo è l'utilizzo di un metodo robusto ed efficiente per calcolare l'inversa di S. utilizzare infatti il metodo (12) per invertire la matrice, oltre ad avere un elevato costo computazionale O(n!), presenta un ulteriore problema: il condizionamento della matrice. Infatti, se la matrice S ha un determinante piccolo, potrebbe portare ad un errore di arrotondamento significativo, generando instabilità numerica.

$$S^{-1} = \frac{1}{\det(S)} \operatorname{adj}(S) \tag{17}$$

Si è deciso, pertanto, di utilizzare il metodo di eliminazione di Gauss-Jordan con pivoting parziale. Questo aiuta a mitigare gli effetti del cattivo condizionamento della matrice e, rispetto al metodo di inversione tramite Laplace, riduce significativamente la complessità computazionale ad $O(n^3)$.
In ultima istanza vengono calcolati lo stato finale (il quaternione corretto), con la sua matrice di covarianza (in questo caso P diminuisce, essendo che la correzione aumenta la confidenza sull'effettivo stato del sensore). Come ultimo step, Il quaternione finale viene normalizzato per la sua norma uno: Questo viene fatto perchè il risultato potrebbe divergere velocemente dato che le quattro componenti dei quaternioni non sono indipendenti tra di loro. Lo schema a blocchi complessivo è visibile in figura 13.

4.2.8 Inizializzazione

Per inizializzare l'EKF, il quaternione iniziale q_0 deve rappresentare correttamente l'orientamento del sistema. La matrice di rotazione **C**, calcolata usando le misurazioni dell'accelerometro e del magnetometro, descrive la relazione tra il sistema locale del sensore e il sistema di riferimento globale. Convertire questa matrice in un quaternione offre una rappresentazione più compatta.
La matrice di rotazione $\mathbf{C} \in R^{3\times3}$ è definita utilizzando:

- $\mathbf{a}_0$: vettore direzione della gravità
- $\mathbf{m}_0$: vettore direzione del campo magnetico terrestre
- $\mathbf{a}_0 \times \mathbf{m}_0$: un vettore ortogonale ai primi due, che completa il sistema di riferimento.

C è una matrice che presenta queste componenti, e la viene normalizzata per evitare errori numerici.
Il quaternione iniziale q_0 viene calcolato a partire dalla matrice **C** usando la seguente espressione:

$$q_0 = \begin{bmatrix} \frac{1}{2}\sqrt{C_{11} + C_{22} + C_{33} + 1} \\ \frac{1}{2}\,\mathrm{sgn}(C_{32} - C_{23})\sqrt{C_{11} - C_{22} - C_{33} + 1} \\ \frac{1}{2}\,\mathrm{sgn}(C_{13} - C_{31})\sqrt{C_{22} - C_{33} - C_{11} + 1} \\ \frac{1}{2}\,\mathrm{sgn}(C_{21} - C_{12})\sqrt{C_{33} - C_{11} - C_{22} + 1} \end{bmatrix}. \tag{18}$$

In questa formula:

- Ogni componente di q_0 è calcolata per garantire che il quaternione abbia una norma pari a 1, preservando le proprietà geometriche della rotazione.
- I termini $\mathrm{sgn}(\cdot)$ determinano il segno corretto delle componenti, evitando ambiguità nella rappresentazione.

La conversione di **C** in q_0 permette di rappresentare la stessa rotazione ma sfruttando i vantaggi del quaternione rispetto al calcolo matriciale. Questa inizializzazione è cruciale per ridurre errori nelle stime successive, che potrebbero portare il filtro a divergere.

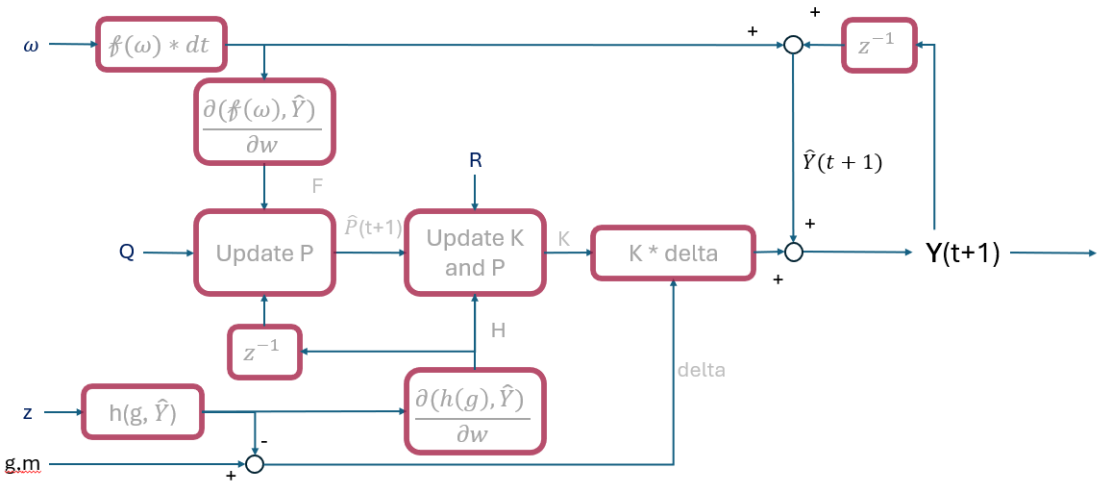


Figure 13: flowchart dell' EKF

4.3. Fall detection

Il processo di rilevamento delle cadute avviene in diverse fasi:

1. **Impatto**: Questa fase rappresenta il momento in cui il ciclista colpisce il terreno o un oggetto. È caratterizzata da una variazione improvvisa e significativa dell'accelerazione, che tipicamente dura tra i 50 e i 200 millisecondi;
2. **Postura**: Dopo l'impatto, la posizione del corpo del ciclista cambia drasticamente rispetto a prima dell'incidente. L'inclinazione della testa può deviare lateralmente o in avanti/indietro, con l'angolo di pitch o roll che supera determinate soglie critiche;
3. **Aftermath**: Dopo l'impatto, il ciclista rimane generalmente immobile per alcuni secondi. L'accelerazione totale tende a stabilizzarsi intorno al valore di 1g (dovuto principalmente alla gravità lungo l'asse z). Se il ciclista fosse cosciente, potrebbe cominciare a muoversi per rialzarsi dopo circa 1-2 secondi, rendendo quindi l'impatto rilevato un falso positivo. Tuttavia, se non viene rilevato alcun movimento entro 10 secondi dall'impatto, l'incidente viene considerato grave.
4. **Allarme**: Se tutte le condizioni sopra descritte sono soddisfatte, il sistema invia un allarme di emergenza.

Per garantire l'accuratezza del sistema, che integra accelerometri, giroscopi e magnetometri, è stata effettuata una pre-calibrazione dei sensori (si faccia riferimento al paragrafo 4.2 per maggiori dettagli) per ridurre errori di bias nei dati e rilevare con precisione l'impatto e i successivi cambiamenti di postura del ciclista.
Studi precedenti [9], hanno evidenziato come l'uso esclusivo di accelerometri possa generare falsi positivi in situazioni come sedersi bruscamente o saltare. Per ridurre al minimo questi errori, il sistema proposto utilizza una combinazione di accelerometri e giroscopi, migliorando così la precisione nel rilevamento delle cadute e riducendo falsi positivi e negativi. Inoltre, come suggerito da Flores et al. [10], per garantire che, in caso di caduta, l'utente riceva soccorso tempestivo, il sistema invia automaticamente un messaggio di emergenza. Questo avviene grazie

all'integrazione del GPS, che permette di trasmettere la posizione esatta del ciclista tramite lo smartphone, insieme al segnale di allarme inviato via Bluetooth ai contatti predefiniti.

Le soglie per determinare l'impatto, l'inclinazione della testa e il tempo di attesa per l'aftermath sono state definite basandosi su studi esistenti, ma sono state adattate ai dati sperimentali ottenuti da noi durante la fase di test. In questo contesto è interessante notare che, mentre esistono numerosi studi sulla rilevazione delle cadute per anziani o durante attività di vita quotidiana (ADL), pochissimi si concentrano sulle cadute in bicicletta. La differenza tra i rischi di una caduta durante un'attività quotidiana e una durante una corsa in bicicletta è significativa, poiché la velocità della bicicletta amplifica la forza dell'impatto, rendendo la caduta molto più pericolosa. Di conseguenza, la ricerca specifica per la rilevazione delle cadute durante le corse in bicicletta è fondamentale. Uno degli articoli di riferimento per la definizione delle soglie [11] ha dimostrato che, in media, un impatto durante una caduta in bicicletta comporta un'accelerazione di 1.3229 g e un angolo di inclinazione della testa di 46.648°. Questo studio ha utilizzato un sensore montato sul casco del ciclista, una configurazione che è stata adottata anche in questo progetto per massimizzare la rilevazione del movimento del casco. È importante notare che questo studio è stato condotto in un ambiente controllato, al chiuso. Pertanto, sono necessari ulteriori esperimenti all'aperto, in condizioni più simili ad una reale pedalata, per verificare l'efficacia del sistema in un contesto più realistico.

L'algoritmo per l'identificazione delle cadute si basa su un macchina a stati. Vi è infatti una variabile globale chiamata "stato" che tiene traccia della condizione in cui ci si trova.
Di base, infatti, la variabile "stato" è nella condizione **"NORMAL"**, in cui si cerca di identificare l'avvenimento di un impatto. Per far ciò viene misurato il delta RMS, una variante del classico RMS (Root Mean Square) che utilizza il delta accelerazioni invece dell'accelerazione stessa. L'RMS classico, fornisce una misura generale dell'intensità del movimento e la formula per calcolarlo è:

$$\text{RMS} = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{19}$$

dove a_x^2, a_y^2, a_z^2 rappresentano le componenti dell'accelerazione lungo i tre assi spaziali.
Nel delta RMS, invece, le componenti dell'accelerazione vengono sostituite con i delta delle accelerazioni, cioè la differenza tra l'accelerazione attuale e quella all'istante precedente:

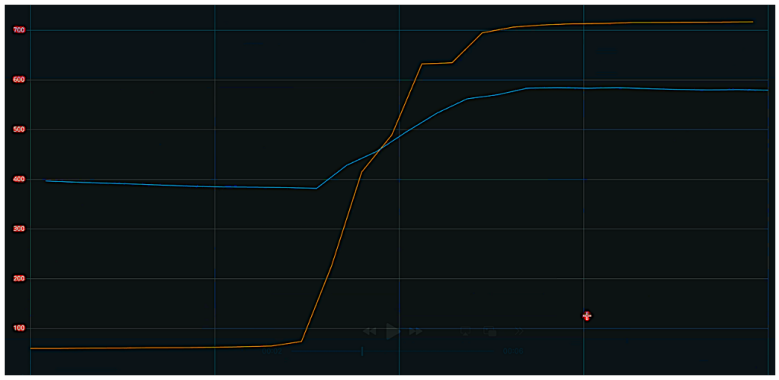
$$\Delta\text{RMS} = \sqrt{(\Delta a_x)^2 + (\Delta a_y)^2 + (\Delta a_z)^2} \tag{20}$$

Sperimentalmente, abbiamo osservato che il delta RMS supera sempre l'RMS nel momento di un impatto, come si può osservare in Figura 14.
Questo perché l'impatto è caratterizzato da un comportamento oscillatorio delle accelerazioni, con variazioni da valori positivi a negativi, che rendono più elevate le differenze tra accelerazioni consecutive rispetto alle accelerazioni stesse. Al contrario, in caso di frenate o accelerazioni intense ma unidirezionali, l'RMS cresce, ma il delta RMS rimane più contenuto, perché le accelerazioni non subiscono variazioni di direzione. Il delta RMS viene inoltre calcolato su finestre temporali di circa 200 ms. Questo intervallo è stato scelto per filtrare piccoli picchi e artefatti, così da evitare che brevi ma intense accelerazioni dovute a irregolarità della strada vengano erroneamente interpretate come impatti significativi. La formula applicata nell'algoritmo per il calcolo del delta RMS è quindi la seguente:

$$\Delta\text{RMS} = \sqrt{\frac{\left(\sum_{i=0}^{N}(\Delta a_x(i))^2\right) + \left(\sum_{i=0}^{N}(\Delta a_y(i))^2\right) + \left(\sum_{i=0}^{N}(\Delta a_z(i))^2\right)}{N}} \tag{21}$$

dove N è il numero di segnali contenuti nella finestra temporale.
Una volta identificato un impatto, la variabile "stato" viene aggiornata a **"IMPACT HIGH ACC"**, e si procede alla ricerca di modifiche posturali. Per far ciò vengono calcolati gli angoli di pitch e roll usando il kalman filter che sfrutta le informazioni provenienti dall' accelerometro e dal giroscopio.
Se uno di questi due angoli supera la soglia critica (*trh ang*) pari a 45°, la variabile "stato" viene aggiornata a **"IMPACT HIGH ANG"** in quanto una modifica posturale è stata identificata. Se invece, nessuno dei due angoli di pitch e roll supera *trh ang* nei 2 secondi successivi all'impatto, allora la variabile "stato" viene riportata alla condizione "NORMAL", in quanto l'impatto era un falso positivo.
Nel caso di modifica posturale si passa a ricercare la stabilizzazione delle accelerazioni tipica della fase di aftermath. Questo viene effettuato sfruttando nuovamente il delta RMS che deve essere circa pari a 0. Ovviamente viene lasciata una piccola tolleranza per il rumore, dunque la stabilizzazione viene individuata se:

$$|\Delta\text{RMS}| < 0.8 \tag{22}$$



(a) RMS in blu e delta_RMS in arancione



(b) RMS in rosa e delta_RMS in giallo. Sono qui mostrate anche tutte le altre accelerazioni (asse x, y, z) con il loro carattere oscillatorio

Figure 14: Confronto tra l'RMS e il delta_RMS al momento di un impatto

La stabilizzazione viene controllata a partire da 5 secondi dopo l'impatto e per i successivi 5-10 secondi. Nel caso risulti presente per tutto il periodo indicato, la variabile "stato" viene aggiornata alla condizione **"IMPACT NO MOVEMENT"**, che genera un allarme.
Lo schema a blocchi dell'algoritmo appena descritto è mostrato in Figura 15.

4.4. Attention

L'algoritmo di attention si propone di individuare situazioni potenzialmente pericolose in cui il ciclista mantiene la testa inclinata con un angolo di pitch $> 20°$ (sia verso l'alto che verso il basso) per un tempo troppo prolungato ($>$3 secondi). La soglia andrebbe aggiustata a seconda dell'inclinazione del casco in base al modello di bicicletta. Nel momento in cui si rileva un evento di distrazione, viene inviata una notifica all'app mobile e viene generata una vibrazione.
Per una maggiore comprensione del flusso di controllo completo si faccia riferimento al diagramma in Figura 16.

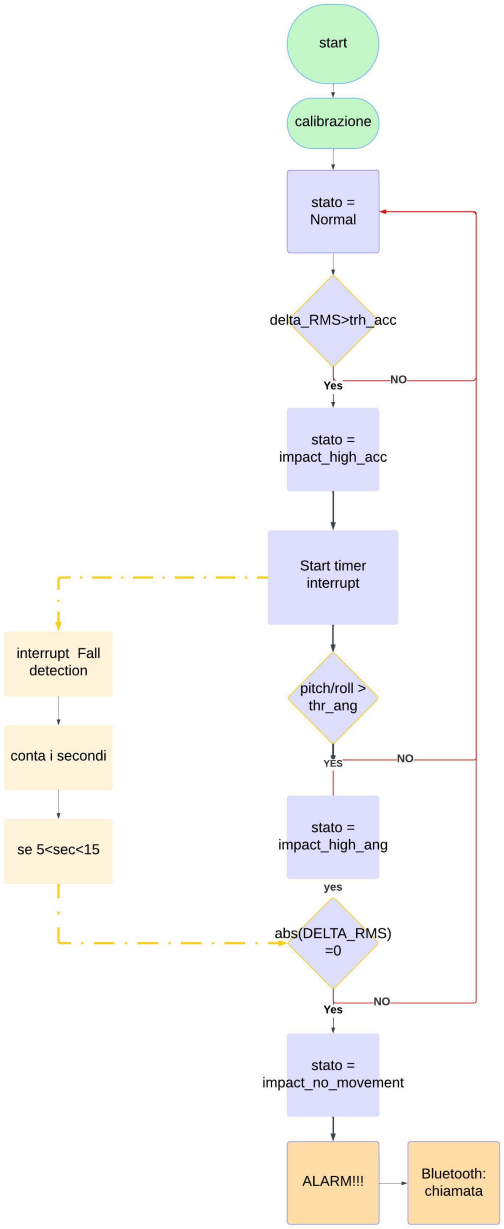


Figure 15: **Schema a blocchi dell'algoritmo di fall detection.**
I blocchi con le frecce tratteggiate si riferiscono alle "azioni" effettuate nell'Interrupt.

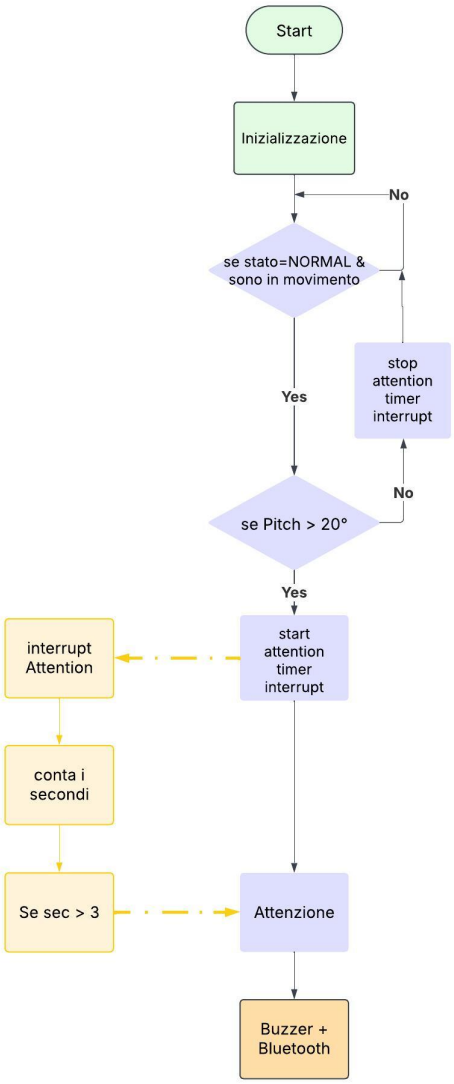


Figure 16: **Schema a blocchi dell'algoritmo di Attention**.
I blocchi con le frecce tratteggiate si riferiscono alle "azioni" effettuate nell'Interrupt.

4.5. TOF

4.5.1 Inizializzazione TOF

Per programmare il sensore TOF VL53L5CX abbiamo utilizzato la libreria *Sparkfun VL53L5CX Arduino Library*, nella quale troviamo diverse classi e funzioni utili per l'inizializzazione del sensore. Nel firmware, a partire dalla classe chiamata SparkFun_VL53L5CX, creiamo un oggetto per ciascun sensore: TOF1 e TOF2. In modo analogo con la classe VL53L5CX_ResultsData inizializziamo una struttura che conterrà i dati di misura: sia le distanze fornite direttamente dal sensore, sia le misure ottenute a seguito dell'applicazione di un filtro SMA.
In seguito all'inizializzazione, definiamo alcune variabili globali che ci serviranno nel firmware per gestire le seguenti operazioni:

- impostare la risoluzione dei sensori
- impostare la frequenza di campionamento del sensore a 10Hz
- gestire il timer dell'ISR con un oggetto della classe SoftwareTimer

Prima di tutto, è necessario riassegnare gli indirizzi dei sensori ToF, in quanto, per impostazione predefinita,

entrambi condividono lo stesso indirizzo 0x29. Un sensore è mantenuto in stato di reset mentre l'altro verrà configurato. La stessa cosa avviene per il secondo ToF. I pad di RST vengono connessi direttamente alle porte GPIO4 e GPIO5 del microcontrollore. Come detto in precedenza, per far comunicare correttamente gli imager prodotti da Sparkfun, è stato necessario eliminare le due resistenze di pull-up da 2.2 kΩ, eccezion fatta per un imager.
Per ciascun sensore viene effettuato un controllo di corretta connessione che, in caso di esito negativo, impedirà l'invio della conferma al dispositivo mobile.
Il primo step riguarda la scelta della risoluzione, con la possibilità di selezionare una matrice 8x8 o 4x4. Per le misurazioni, si è deciso di adottare sensori con matrice 8x8, così da ottenere dati più accurati e dettagliati.
Successivamente, la frequenza di campionamento viene fissata a 10 Hz, trovando un equilibrio tra accuratezza e costo computazionale. Viene inoltre configurato un interrupt associato a un timer, che viene attivato ogni 50 ms, garantendo una frequenza di campionamento stabile e coerente.
Completate tutte le inizializzazioni, si procede con l'abilitazione dell'acquisizione dei dati per ciascun sensore.

4.5.2 Campionamento TOF

Il timer è quindi configurato con una frequenza di 20 Hz al fine di garantire il rispetto della frequenza di campionamento di ciascun sensore. Di conseguenza, la Interrupt Service Routine (ISR) viene eseguita due volte ogni 100 ms, consentendo l'acquisizione delle misurazioni a intervalli regolari. Prima dell'invio dei dati tramite I2C, il microcontrollore aspetta la comunicazione del sensore che i dati siano effettivamente pronti. Se risulta vero, si procede con l'acquisizione vera e propria ed il salvataggio di una matrice 8x8 al tempo attuale e al tempo precedente. La matrice dovrà essere indicizzata per tener conto della struttura specchiata come riportato in Figura 17.

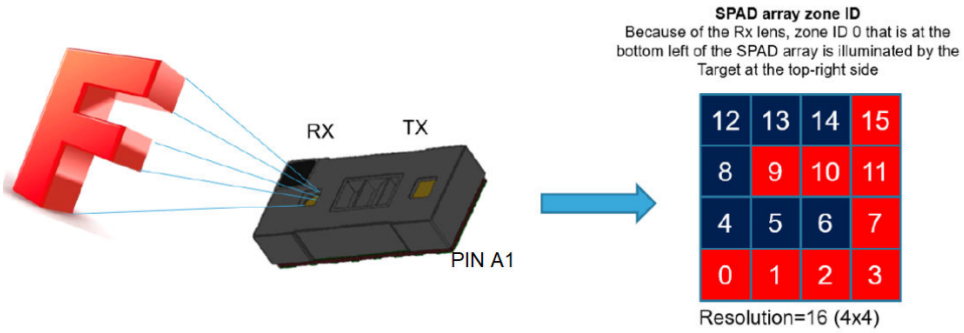


Figure 17: Immagine tratta dal datasheet del sensore[7]. Si nota che le misure di distanza acquisite devono essere invertite.

4.5.3 Calcolo distanza

Dopo aver acquisitivo correttamente le misure di distanza rilevate da ciascuno dei 64 pixel del sensore ToF, procediamo all'implementazione di un **filtro Simple Moving Average** con finestra di 3 campioni. Quest'ultimo è un metodo di smorzamento dei dati che calcola la media aritmetica di un determinato numero di valori consecutivi nel tempo, fornendo una stima più stabile dei dati in ingresso. In questa applicazione, l'SMA è utile per ridurre il rumore e le fluttuazioni rapide delle misurazioni, migliorando l'affidabilità dei risultati e offrendo una lettura più accurata della distanza media nel tempo.

4.5.4 Avviso superamento soglia

Per rilevare la presenza di un veicolo, si è deciso di utilizzare soltanto il valore di distanza tra il casco ed eventuali ostacoli. Questa decisione è stata presa principalmente a seguito di una cattiva performance da parte del primo metodo utilizzato, ossia della velocità relativa. La scelta più intuitiva infatti, sarebbe ricaduta sul calcolo della velocità relativa tra il ciclista e un possibile veicolo, avendo accesso alla distanza ed alla finestra temporale intercorsa tra le varie acquisizioni. Tuttavia, a causa dell'elevato rumore sui segnali, questa opzione non ha portato a risultati affidabili. Si è preferito dunque utilizzare la distanza relativa ed, in particolare, la soglia è stata imposta ad 1.5 metri, per rispettare le normative vigenti di distanza tra veicolo e velocipedi durante la manovra di sorpasso (art.148 del Codice della Strada Italiano).
Per stabilire se il sensore abbia effettivamente rilevato un ostacolo ad una distanza inferiore rispetto a quella predefinita, ci si avvale del seguente protocollo:

1. per ogni istante di acquisizione, si individua il valore minimo salvato nella matrice 8x8.
2. si verifica se tutti gli 8 pixel adiacenti superano anch'essi la soglia.
3. se la condizione precedente è verificata, si attiva lo stato di allarme ed una notifica viene inviata tramite BLE all'app mobile. In caso contrario invece, significa che il minimo trovato è solo frutto di componenti rumorose oppure che il veicolo rilevato è successivamente distante.

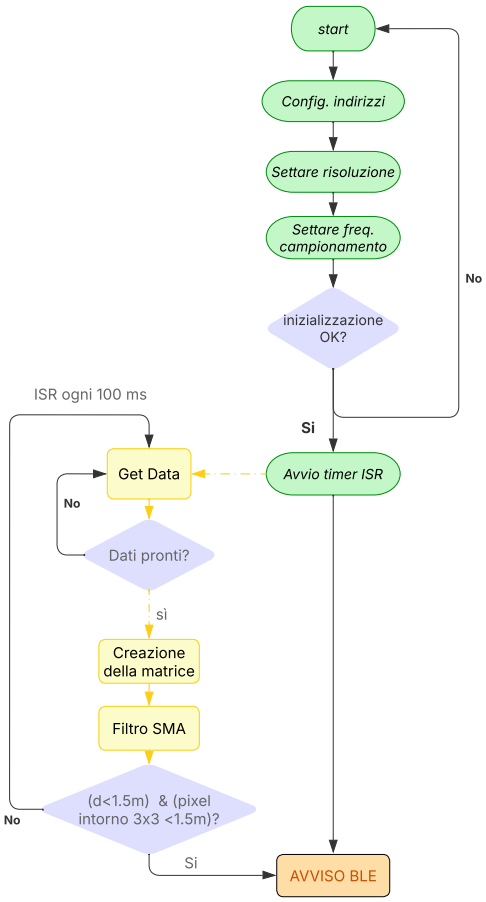


Figure 18: Diagramma di flusso TOF

4.6. Bluetooth Low Energy

Il Bluetooth Low Energy è una tecnologia wireless progettata per ridurre il consumo energetico rispetto al Bluetooth classico. In questo progetto è stato creato un servizio BLE personalizzato, con cinque caratteristiche:

- Proximity
- Fall
- Distraction
- RequestMagneticData
- MagneticData

Le prime quattro utilizzano le Indication GATT Indicate, ovvero che il server manderà un messaggio al client iscritto alla caratteristica, il quale deve confermarne la ricezione con un ACK (conferma). Se il client non conferma, il server può gestire un'eventuale ritrasmissione del messaggio per garantire l'affidabilità della comunicazione. L'ultima caratteristica (MagneticData) è impostata con proprietà di scrittura (write), quindi il client può inviarle dati. Nel setup() viene inizializzato il BLE, impostando la potenza di trasmissione e il nome del dispositvo. Poi vengono attivate le callback di connessione e di disconnessione. Una volta fatto ciò viene iniziato il servizio e le 5 caratteristiche, con degli UUID (Universally Unique Identifier, identificatore univoco universale) e le propietà sopra citate. Viene inoltre attivata la callback per l'arrivo dei dati necessari per la definizione del vettore di campo magnetico di riferimento globale, necessario per lo step di correzione dell' EKF. Il microcontrollore a questo punto l'advertising fino a quando non rileverà un dispositivo connesso. L'advertising avviene a intervalli specifici, iniziando in modalità veloce per aumentare la probabilità di rilevamento. In caso

di disconnessione il microcontrollore ricomincierà l'advertising.

5. Software

Per semplificare l'utilizzo del caschetto mentre si va in bicicletta, si è deciso di utilizzare un'interfaccia utente tramite smartwatch. Esso comunicherà con un cellulare dal quale sarà possibile accedere a funzionalità più avanzate come la visualizzazione delle statistiche e la posizione di un contatto che ha mandato l'SOS. Come prototipo è stata implementata un'applicazione con Android studio che permette di gestire tutto dal telefono, non avendo a disposizione uno smartwatch su cui testarla.

5.1. Registrazione e login

Al primo accesso all'applicazione, compare una schermata di registrazione (Figura 19a), nella quale l'utente può inserire la propria mail e password. Inoltre gli verranno richiesti il numero di telefono e l'indirizzo email di un contatto di emergenza. Una volta premuto *"Registrati"*, se tutte le informazioni sono state inserite correttamente, verrà creato un nuovo account su Firebase Authenticator, e verranno salvate le informazioni del contatto di emergenza su Cloud Firestore. Nel caso in cui l'utente abbia già un account, potrà passare a una schermata di Login in cui effettuerà l'accesso(Figura 19b).
La prima volta che viene utilizzata l'applicazione verranno chiesti all'utente tutti i permessi necessari per l'utilizzo del Bluetooth Low Energy, delle chiamate e della geolocalizzazione. Agli utilizzi successivi si aprirà direttamente la pagina principale, nella quale viene verificato in background che non sia stata cambiata qualche autorizzazione, e solo in tal caso viene richiesto il permesso(Figura 19d).

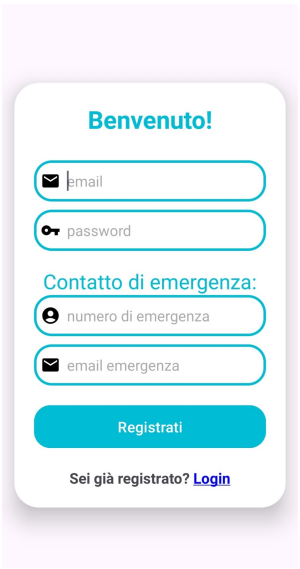
5.2. Homepage e funzionalità principali

Dall'Home Page si può telefonare al contatto di emergenza, entrare nelle impostazioni, accedere alla pagina con le statistiche e gestire il Bluetooth Low Energy. In particolare, il plulsante con il simbolo del Bluetooth è verde quando il dispositivo è connesso al caschetto e rosso se è disconnesso. Premendo questo pulsante quando è verde il caschetto viene disconnesso, mentre quando è rosso viene avviata la ricerca BLE.
Una volta effettuata la ricerca l'orologio mostrerà tutti i dispositivi vicini che mandano in advertisement l'UUID del servizio GATT creato dal caschetto.(Figure19i) L'utente selezionerà il dispositivo corretto, che verrà memorizzato per effettuare la connessione automatica nei successivi accessi all'applicazione. Una volta selezionato il dispositivo, l'orologio si sottoscrive alle 4 caratteristiche (caduta, macchina in avvicinamento, distrazione dell'utente e richiesta di invio delle componenti del campo magnetico) con il metodo "indicate", che fa arrivare una notifica al client GATT ogni volta che il dato cambia. Questo manda un acknowledgment al microcontrollore che, in automatico, gestisce un'eventuale ritrasmissione del dato in caso di mancata consegna.

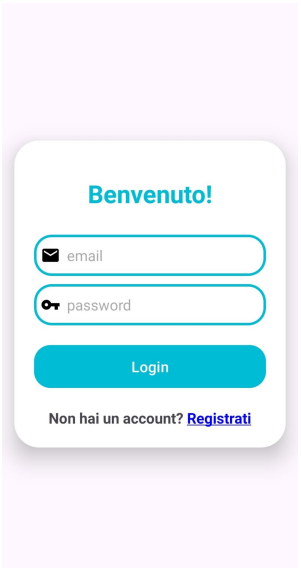
Se lo smartwatch riceve una messaggio BLE di macchina vicina, di distrazione dell'utente o di caduta, vibra per attirare l'attenzione dell'utente e mostra una schermata per ciascuno di questi eventi. (Figure19e, 19f, 19g) In particolare, in caso di caduta si apre una schermata con un conto alla rovescia di 30 secondi. Se l'utente non lo disattiva, al termine del countdown parte automaticamente una chiamata di emergenza al contatto fornito in precedenza. Questo meccanismo, permette di disattivare l'invio della chiamata in caso di falso positivo. In caso di vera caduta invece, partirà la chiamata e verranno inoltre salvate su Firestore Database le coordinate del punto in cui si trova il ciclista, in modo tale che il contatto di emergenza possa, entrando nella propria app, visualizzare il punto esatto in cui la persona cara è caduta e, se necessario, provvedere a chiamare i soccorsi(Figura 19k).

Per ottenere le coordinate esatte del ciclista, il microcontrollore invia periodicamente una richiesta allo smartwatch per ricevere i dati GPS, come latitudine e longitudine. Queste informazioni vengono poi confrontate con un file CSV contenente le coordinate di tutta Europa, che consente di mappare la posizione del ciclista o di associare i valori di intensità di campo magnetico misurati nel sistema di riferimento assoluto in quel luogo.

Inoltre, ogni volta che il microcontrollore manda una notifica di caduta, di macchina vicina o di distrazione dell'utente, viene salvato il numero di questi eventi su Cloud Firestore, organizzandoli per utente e data. Grazie alle regole di sicurezza di firebase, ogni utente potrà vedere solamente i propri dati. Inoltre, vengono aggiornate delle variabili globali che tengono traccia del numero totale di cadute, distrazioni, macchine vicine rilevate e "sessioni", ovvero i giorni in cui è stato usato il caschetto, per poi essere visualizzate nell'applicazione. Dall'homepage, infatti, è possibile accedere alla pagina con le statistiche, dove l'utente può consultare un grafico



(a) Registrazione



(b) Login



(c) Impostazioni



(d) Homepage



(e) Macchina



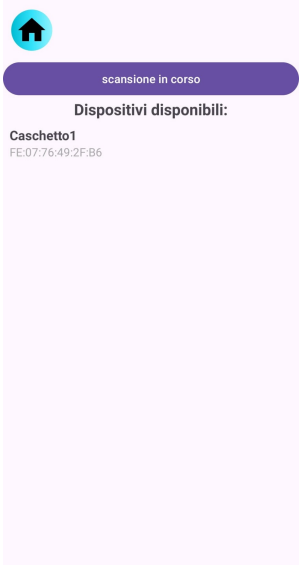
(f) Distrazione



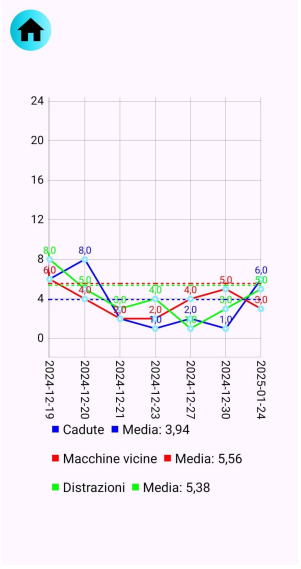
(g) Caduta



(h) Disconnessione



(i) Ricerca dispositivi



(j) Statistiche



(k) Mappa

Figure 19: Schermate dell'applicazione

che mostra il numero di cadute, distrazioni e macchine vicine che sono state rivelate durante le sue sessioni, confrontandoli con la media di tutti gli utenti (Figura 19j). Infine nelle impostazioni è possibile modificare il numero di emergenza o cambiare account (Figura 19c).

5.3. Gestione disconnessioni

Il microcontrollore e lo smartwatch dovranno inoltre occuparsi di gestire in maniera intelligente un'eventuale disconnessione, segnalandola all'utente e provando a riconnettersi.
In particolare, in caso di disconnessione il microcontrollore ricomincia l'advertisement, mentre l'orologio vibra per notificare l'utente e intanto prova a riconnettersi, fino ad un massimo di cinque tentativi, distanziati di tre secondi. Anche in questo caso si apre una schermata con un conto alla rovescia di 40 secondi, che può essere disattivato in due modi: manualmente dall'utente, che conferma di aver spento lui il casco, oppure dalla riconnessione stessa (Figura 19h).
In caso contrario, al termine del countdown, viene attivata automaticamente la chiamata di emergenza, in quanto si presuppone che sia avvenuto un incidente ed abbia danneggiato il casco.

6. Visualizzazione

Durante il periodo di implementazione del dispositivo ci si è avvalso di uno strumento di visualizzazione 3D dei dati. E' stata creata un'interfaccia utilizzando Python che potesse mostrare 2 dati:

- **Posizione rispetto alla terna di riferimento**: il blocco viola in Figura 20 risulta solidale con le due finestre e ruota seguendo i valori in uscita dall'EKF.
- **Heat map dei due ToF**: i pixel che rilevano distanza brevi, saranno mostrati su una scala di rosso, mentre quelli più distanti, su una scala di verde.

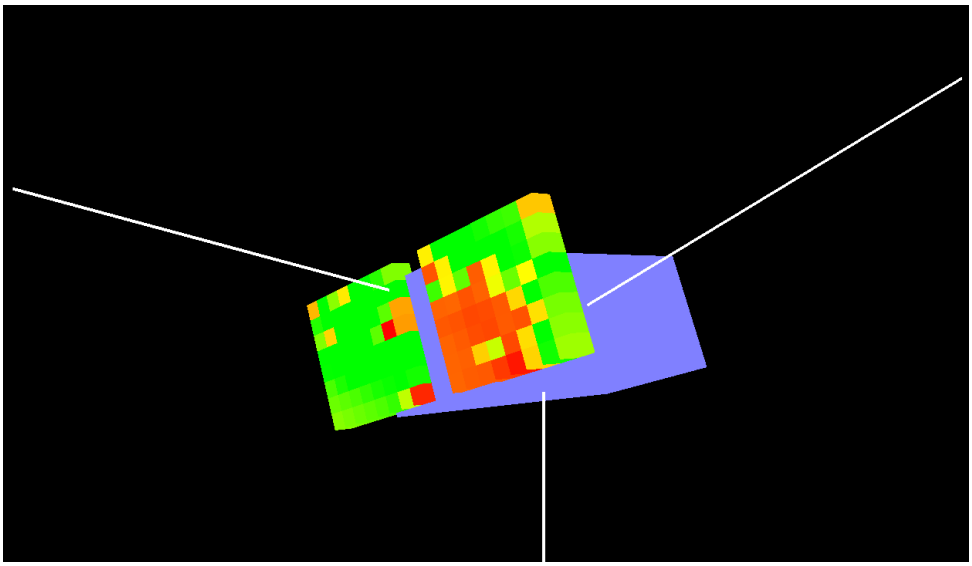


Figure 20: Visualizzazione della simulazione dinamica. In bianco gli assi di riferimento.

7. Protocollo di validazione

Il progetto implementato non ha funzionalità strettamente mediche; per questo motivo non si è pensato ad un processo di validazione vero e proprio ma di seguito viene riportato un ipotetico approccio metodico basato su test sperimentali, analisi comparative e feedback degli utenti.

7.1. Validazione della Rilevazione delle Cadute

Per garantire la precisione del sistema nel rilevare cadute, vengono impiegate diverse metodologie:

- Test in laboratorio: simulazione di cadute controllate con un manichino dotato di caschetto e confronto

dei dati acquisiti con soglie di accelerazione e variazioni di orientamento, sulla base di modelli di caduta noti in letteratura.
- Test con ciclisti: Far indossare il caschetto a ciclisti in ambienti sicuri e simulare cadute su superfici diverse (asfalto, sterrato, erba) per valutare la sensibilità del sistema.
- Confronto con dispositivi certificati: Comparare le rilevazioni del caschetto con un accelerometro di riferimento per verificare la correttezza dei dati raccolti.

7.2. Validazione del Rilevamento dei Veicoli

- Test in ambiente controllato: posizionare il caschetto su un supporto fisso e avvicinare veicoli a diverse velocità e angolazioni, verificando la capacità del sistema di rilevarli correttamente.
- Test su strada: Far indossare il caschetto a ciclisti in scenari reali e misurare la precisione del rilevamento dei veicoli confrontando i dati con un radar, una telecamera esterna oppure una sensore generico di distanza.
- Test di latenza: Valutare il tempo di risposta del sistema dall'individuazione del veicolo fino alla segnalazione al ciclista.

7.3. Test di Robustezza e Affidabilità

- Test in diverse condizioni meteo: Valutare il funzionamento del caschetto sotto precipitazioni, vento o estrema luminosità ambientale.
- Test di durata: sottoporre il sistema a lunghi periodi di utilizzo per verificarne la stabilità e il consumo energetico.
- Test di resistenza agli urti: Assicurarsi che il dispositivo mantenga la funzionalità anche dopo impatti e vibrazioni. Questo aspetto è particolarmente rilevante nell'ambito studiato poichè proprio a seguito dell'impatto risulta fondamentale il corretto funzionamento del dispositivo, il quale deve inviare messaggio di soccorso.

7.4. Feedback Utente

- Test con ciclisti reali: identificare un campione di ciclisti e raccogliere feedback su comfort, usabilità e affidabilità delle segnalazioni.
- Analisi dei falsi positivi e negativi: identificare situazioni in cui il sistema potrebbe generare avvisi errati o non rilevare correttamente per ottimizzarne le prestazioni.

Combinando questi test, si può verificare se il caschetto soddisfa i requisiti di sicurezza e affidabilità prima della sua commercializzazione.

8. Conclusioni

Questo progetto ha portato alla realizzazione di un casco che migliora la sicurezza dei ciclisti, affrontando il problema della distrazione e del ritardo nell'intervento di soccorso. Grazie all'integrazione dei sensori IMU e ToF, è stato possibile monitorare sia la distrazione del ciclista sia quella dei veicoli circostanti, con l'obiettivo di prevenire collisioni e rilevare eventuali cadute.
Il sistema è stato progettato per inviare un allarme in caso di emergenza, migliorando il tempo di risposta dei soccorsi grazie alla localizzazione GPS e alla comunicazione tramite BLE. Il prototipo sviluppato ha dimostrato di essere efficace nell'individuare situazioni di pericolo e di supportare il ciclista in modo tempestivo.
Nonostante il successo, il dispositivo potrebbe essere migliorato nei seguenti aspetti:

- Utilizzare un diverso sensore di distanza, più performante rispetto al TOF qui utilizzato. Infatti, questo sensore è risultato essere troppo sensibile alla luce solare e quindi non utilizzabile durante le ore diurne, rendendo il dispositivo poco adattabile. Inoltre, nonostante la distanza di rilevamento teorica fosse di 4 metri, nella realtà è risultata essere molto minore, rendendo quindi il dispositivo poco adatto per il suo scopo;
- Integrare anche il terzo sensore di distanza, per avere una visione più completa del retro del ciclista;
- Espandere l'algoritmo di attention anche per l'angolo di yaw, evitando così che il ciclista si guardi in torno per troppo tempo distraendosi dalla strada. Questo potrebbe essere fatto mediante l'utilizzo di reti neurali ricorrenti leggere con celle LSTM (Long-short Term Memory), [12] oppure mediante la loro versione maggiormente efficiente, le GRU (Gated Recursive Unit) [13]. questo permetterebbe di discriminare correttamente un evento di rotazione della testa da una semplice curva. Occorre tuttavia procedere con

un' acquisizione di un dataset contenenti tali eventi e annotarli. Ciò non è stato possibile dato che prototipo funzionante in maniera robusta è stato ultimato nelle ultime due settimane di lavoro;
- Integrare nella mobile app la possibilità di settare delle soglie rispetto alla distanza e velocità alla quale l'avviso viene mandato;
- Introdurre la funzionalità di inviare un SMS con le coordinate GPS al contatto d'emergenza appena la chiamata di SOS è inoltrata. Al momento infatti le coordinate vengono salvate su Cloud Firestone, e il contatto di emergenza deve entrare nella sua app per visualizzarle, cosa che rallenta il soccorso. Tuttavia, la funzionalità di mandare SMS non è stata fin ora implementata in quanto a pagamento;
- introdurre l'allarme acustico tramite buzzer, cosa non effettuata per limiti di tempo, in quanto all'ultimo è stato notato che i buzzer già in possesso che si pensava di usare, richiedevano in ingresso una tensione superiore a quella erogata dal microcontrollore.

In futuro, sarebbe interessante estendere il progetto a scenari extraurbani dove le condizioni di traffico e le situazioni di pericolo sono differenti. Un ulteriore passo potrebbe consistere nella realizzazione del casco su larga scala, per renderlo accessibile a un numero maggiore di ciclisti, contribuendo così a una maggiore sicurezza sulle strade.

References

[1] UE. European road safety observatory. facts and figures. buses / coaches / heavy goods vehicles - 2020, 2018. PDF.

[2] UE. Ciclisti ed incidenti stradali. analisi e monitoraggio degli incidenti stradali relativi ai ciclisti in regione lombardia, 2018. PDF.

[3] Istat. Incidenti stradali. anno 2022, 2023. PDF.

[4] A. Khan, F. Bibi, M. Dilshad, S. Ahmed, Z. Ullah, and H. Ali. Accident detection and smart rescue system using android smartphone with real-time location tracking. *International Journal of Advanced Computer Science and Applications*, 9(6):341–355, 2018.

[5] Datasheet del microcontrollore nrf52840, 2025. Link per il datasheet.

[6] Datasheet del sensore imu icm-20948. Link per il datasheet.

[7] Datasheet del sensore sparkfun qwiic tof image mini vl53l5cx. Link per il datasheet.

[8] Motioncal calibration tool. Link per la pagine web.

[9] Q. Li, J. A. Stankovic, M. A. Hanson, A. T. Barth, J. Lach, and G. Zhou. Accurate, fast fall detection using gyroscopes and accelerometer-derived posture information. In *2009 Sixth International Workshop on Wearable and Implantable Body Sensor Networks*, pages 138–143, 2009.

[10] J. Flores, C. Cruz, L. Rolon, L. M. H. López, A. Rios, I. Olivari, and I. J. Vergara-Laurens. Rapid response fall detection system. In *Global Partnerships for Development and Engineering Education: Proceedings of the 15th LACCEI International Multi-Conference for Engineering, Education and Technology*, pages 1–6, 2017.

[11] Y.-S. Yang N.-B. Lee H.-R. Choi, M.-H. Ryu and D.-J. Jang. Evaluation of algorithm for the fall and fall direction detection during bike riding. *Department of Healthcare Engineering, Chonbuk National University*, 2017.

[12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9:1735–1780, 11 1997.

[13] Kyunghyun Cho, Bart van Merrienboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches, 2014.